Sets Are Rules, Not Buckets: Intensional Programming in C++23 via `dedekind`*

The `dedekind` Authors

April 29, 2026

Abstract

Named algebraic structure at the type level, discharged by the compiler, narrows the gap between abstract mathematics and machine code without a separate proof assistant. The underlying theory is not new—ETCS [2], Wadler's parametricity and propositions-as-types reading [3, 4], and proof-assistant-grounded formalisation in Coq, Agda, and Lean have been studied for decades. What is new is the *cost-benefit calculus*: modern transformer-based coding assistants substantially compress the time and expertise needed to author opt-in trait specialisations, axiom-shape concepts, and the surrounding library scaffolding, turning what was an academic specialism into an actionable workflow for working software engineers. The meta-claim is not ours to originate; we make it here so the body's executable exhibit can be read in that light.

We present `dedekind` [5], an open-source C++23 library (Apache-2.0), as that exhibit. It embeds formal set-theoretic and categorical structures directly into the type system and targets *mainstream C++23*—concepts, non-type template parameters, named modules—rather than a research compiler. The design rule is *intensional first, realize when you mean it*: a set is represented as a predicate $P : A \to \Omega$ over an ambient species A, not as a heap-allocated container, with two practical consequences:

- Set operations (intersection, complement, union) compose as predicate combinators at compile time, enabling the LLVM backend to perform *structural pruning*: logically empty intersections collapse into zero-instruction machine code.
- Numeric realisation is deferred to explicit boundaries, cleanly separating abstract algebraic reasoning (e.g. over the naturals $\mathbb{N}$) from hardware-constrained IEEE 754 floating-point [6] semantics.

The paper identifies *Axiomatic Systems Programming* as a nascent paradigm in which the compiler is a structural gate, not a theorem prover, and demonstrates the discipline through a textbook linear-programming instance that reduces at compile time to its optimal vertex as a typed constant; the same reduction over a dual-number ring additionally recovers forward-mode automatic differentiation [7] as a sibling compile-time collapse, and the discipline extends to a Fundamental Theorem of Calculus (Part II) discharged inside a `static_assert`.

Keywords: C++23; concepts; non-type template parameters; compile-time verification; Axiomatic Systems Programming; type-directed collapse; linear programming; forward-mode automatic differentiation.

*Draft intended for submission to *Programming* [1], the journal for programming-related research. The paper's scope is deliberately narrower than the companion project report: this document focuses on the sets-as-predicates thesis and compile-time structural pruning; the broader library (categorical foundations, algebra tower, numeric tower, geometry, linear algebra, morphologies, Python facade) is documented in the companion Report distributed alongside the source tree.

1 Introduction

1.1 Motivation: Mathematical Faithfulness in a Compiled Library

A library implementer who wants to expose mathematical abstractions in a compiled language faces a recurring tension. Algebraic structures—groups, rings, fields, vector spaces—are defined by laws: associativity, commutativity, identities, inverses. But the concrete carriers a compiled language offers violate those laws conditionally: a 32-bit signed `int` is not an additive group (overflow, `INT_MIN` has no negation), and IEEE 754 `double` [6] is not associative under addition. The library author's options are to ignore the divergence (silent bugs on specific inputs), police it at runtime (hot paths pay), or encode it in the type system so the compiler can police and exploit it. This paper is about what happens when a library takes the third option seriously, in standard C++23.

The third option requires a type system strong enough to name algebraic structure and to discriminate between carriers that satisfy a law and those that do not. Haskell type classes [8] and dependently-typed languages such as Coq and Agda [9] demonstrate the expressiveness; both impose runtime or toolchain costs that are hard to justify when the library's consumers are writing systems code. C++23 now offers a pragmatic middle ground: *concepts* (the compile-time analogue of Haskell type classes) check structural membership at compile time without nominal declarations, *non-type template parameters* (NTTPs—template arguments that are values, not types) let lambdas and other compile-time data appear in type signatures, and *named modules* enforce a strict directed-acyclic build graph. The compiler becomes both a *verification gate*—violating an algebraic law is a type error, not a runtime exception—and an *optimization engine*—the LLVM backend exploits the structural knowledge inherited from the discharged proof obligations.

Our posture is explicit: `dedekind` is a systems-programming library that treats mathematical concepts as first-class compile-time objects. The library is implemented in C++23, and the compile-time concept machinery is its primary interface; the consumer we design for is another C++ author who would otherwise bolt ad-hoc structure onto unverified numeric types. Choosing to land the semantics at the compile-time layer—rather than as runtime abstractions, or as a DSL lowered to a runtime solver—is a design commitment that shapes what the compiler can do for the user and what it cannot.

Thesis. In one sentence: *named algebraic structure at the type level, discharged by the compiler, narrows the gap between abstract mathematics and machine code without a separate proof assistant.* Every section of the paper serves that sentence. Sets-as-rules, intensional-first, computability-tier collapse, the LP and AD and FTC reductions—these are the *means* by which the thesis is discharged, and the mechanical witnesses (`static_asserts` and IR fixtures) are the evidence that it is discharged in practice.

What is new. Compile-time verification of algebraic structure is not new in the abstract. Agda and Coq host it natively; Liquid Haskell attaches refinement predicates to types; C++ template-metaprogramming traditions (most visibly Boost.Hana [10] and CTRE [11]) have demonstrated type-level computation at scale for over a decade; autodiff libraries such as Ceres-Solver [12] and ENOKI [13] have carried fragments of the dual-number story into production C++. What this paper adds is an end-to-end exhibition of the technique—named algebraic structure, discharged by the compiler, with the LLVM backend carrying the discharge through to optimised machine code—in standard C++23, without a proof assistant, a custom type checker, or a bespoke language extension. `dedekind` is the existence proof. The viewpoint on which it rests we name *Axiomatic Systems Programming*: an engineering discipline of practising, not merely describing, type-level structural verification in mainstream compiled code. The discipline can be lifted, with modest adjustment, to any language whose type system can name a predicate over compile-time data and check it mechanically (concepts-style Java/C#, strictly-typed Python, the relevant parts of Rust). The contribution of this paper is an articulation of that discipline, together with a worked library that discharges its claims mechanically.

Intended target. The discipline lands well in CPU-first and embedded machine-learning deployments—small fixed-topology problems with hard runtime-footprint constraints—where the technique's compile-time-substrate posture sits one layer below existing embedded ML runtimes such as TFLite Micro [14].

The compiler as a poor man's theorem prover. A pedagogical framing that runs through this paper: we are using `clang++` as a poor man's theorem prover. The compiler is not searching for proofs—it has no tactic language, no resolution procedure, no proof assistant in the loop. What it *does* have is a type system strong enough to name a predicate, a `static_assert` strong enough to discharge it, and an LLVM backend that will happily remove what has already been decided. The library author supplies narrow, mechanically checkable proof obligations; the compiler discharges them; the optimiser then deletes the code that would have re-checked them at runtime. As a welcome side effect, the same machinery produces a family of composable library combinators (set meets, cuts, halfspaces, LP reductions) whose laws are verified at translation time and whose trivial cases collapse to zero instructions.

Most of what follows is therefore *not* specific to C++. Concepts have close analogues in Java (sealed types plus generic constraints), C# (constrained generics, augmented by Roslyn source generators and analysers), and Python (PEP 484 type hints under a strict `mypy/pyright` pass). Any language whose type system can name a predicate over compile-time data and check it mechanically is a candidate host for the same technique. What C++23 contributes on top is the LLVM backend sitting downstream of the type checker: discharged proofs do not only *verify*, they also *optimise*, and the connection between the two is what lets the LP vertex collapse to a literal return rather than to a skipped runtime check.

What this buys, concretely. From the design rule ("name the structure in the type, let the compiler discharge the laws") and a small handful of primitives (concepts, NTTP-carried predicates, halfspaces, cuts), the library builds, at compile time, a numeric tower ($\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$ via Dedekind cuts, $\mathbb{C}$, dual numbers), a small linear-algebra layer, a calculus layer (numerical / polynomial / dual-number AD), and an optimisation layer (the LP vertex enumerator of Section 5.2). No runtime solver ships with any of it. The centrepiece (Section 5.2) is a textbook LP that reduces at compile time to its optimal vertex as a typed constant; the same reduction over the dual-number ring (Section 5.3) recovers forward-mode automatic differentiation as a sibling compile-time reduction in the sense of Elliott [7].[1]

The three claims this paper substantiates.

1. Sets are *rules* (predicates), not *buckets* (containers).
2. Computation is *intensional until explicitly realized.*
3. The compiler is used as a *verification and optimization engine* for algebraic invariants.

The remainder of the paper develops these three claims and demonstrates each by a reduction that compiles to a constant.

[1]The discipline extends to polynomial calculus: the formal derivative of a polynomial is a `static_assert`-checked identity, and the Fundamental Theorem of Calculus (Part II) is discharged at translation time as numerical agreement between a trapezoidal quadrature and a closed-form antiderivative. Worked exhibits in the companion repository's `dedekind.algebra:polynomial` and `dedekind.analysis:ftc` partitions; `static_assert` witnesses pin $\frac{d}{dx}P(x) = P'(x)$ on polynomials over $\mathbb{Z}$ (and, where a divisible carrier is supplied, over $\mathbb{Q}$) and pin agreement within a documented tolerance for the FTC bridge.

Intensional sets, structurally. Where this paper says *intensional set*, the intended referent is the structuralist set in the sense of the *nLab* community [15]: an object whose membership is given by a characteristic morphism into a subobject classifier, situated in a Cartesian-closed ambient. The standard categorical fact is that every ETCS category is a Cartesian-closed category, but most Cartesian-closed categories are not. The library now enforces the subsumption directly: an object satisfying IsSet entails the CCC witness on its ambient species, so any ETCS-set carrier — the finite field $\mathbb{F}_{64}$, the cyclic ring $\mathbb{Z}/N\mathbb{Z}$, the rationals $\mathbb{Q}$ — comes with 1, $\times$, and $(-)^{(-)}$ for free. What this buys is the Lambek–Scott correspondence [16]: a Cartesian-closed category *is* a simply-typed lambda calculus, and once the subsumption is wired into the concept, `dedekind`'s set-builder DSL is one — function spaces, currying, evaluation, and product/coproduct introduction are witnessed by the same type-system machinery that discharges the algebraic laws of Section 3.4.

What else falls out for free. Naming the structure in the type buys non-trivial mileage at no runtime cost. Three quick examples illustrate the pattern: (i) the scalar / column / row / square family $\left(T, T^2, {}^tT^2, M_2(T)\right)$ is functorial in T and each functor is an IsFunctor witness, so an arrow $f : T \to T$ lifts elementwise without per-carrier boilerplate; (ii) the scalar embedding $T \hookrightarrow M_2(T)$ is the monadic unit η_F of the shape functor, $\eta_{M_2}(s) = s \cdot I_2$, pinned as a witness next to the carrier; (iii) the LP reduction of Section 5.2 is a co-Kleisli arrow $\varepsilon_{\mathrm{LP}} : \mathrm{Polytope}(T) \to T^2$ extracting the optimal vertex from the polytope context.[2]

1.2 How to Read This Paper

Code and mathematics appear side by side throughout. That pairing is the whole point. Most of the interesting objects in this paper—Dedekind cuts, rings, dual numbers, polytopes, active sets, ring homomorphisms— have two faces: a math notation most readers have met at some point, and a C++ type most readers will find easier to trace. Put both on the page and you get a kind of binocular vision: whichever face is clearer for you carries the meaning, and the other face falls into place.

No prior familiarity with C++23 templates or with the specific mathematical objects is required. When something non-obvious appears—a non-type template parameter, a ring homomorphism, a co-Kleisli arrow—it is unpacked on first use, often by showing the corresponding C++ idiom next to the math definition and inviting the reader to read them as two names for the same thing. If you are at home with Haskell's type classes, Agda's dependent types, or Coq's proof terms, many of the moves will feel familiar; if not, the C++ side is the on-ramp.

1.3 Paper Structure

Section 2 motivates the choice of C++23 as the implementation substrate and introduces the *Axiomatic Systems Programming* paradigm. Section 3 explains the intensional-first design rule and the tension between abstract algebraic numerics and IEEE 754 hardware floating point. Section 4 develops the view of sets as predicates over ambient species and shows how set operations compose symbolically. Section 5 presents compile-time structural reductions: halfspace pruning, linear programming as a reduction to a typed vertex, and forward-mode automatic differentiation as a sibling reduction over the dual-number ring. Section 6 surveys related work. Section 7 concludes.

[2] The companion claims compose through the Kleisli machinery already in the library (the Path comonad, the Box/Maybe monads), with bind ($\gg=$) and extend ($\ll=$) operators. The categorical vocabulary is not decorative: it lets the same primitives carry several theorem discharges—elementwise lifting, scalar-into-matrix monomorphism, LP counit—each as a typed constant in the emitted object file where elsewhere it would cost a tactics file and a separate compilation pipeline.

2 Axiomatic Systems Programming

2.1 The Substrate: C++23

The *two-language problem* [17] is the observation that research code is written in an expressive high-level language (Python, R, Julia) while performance-critical execution is delegated to a separately maintained C or C++ backend. Bridging tools such as `nanobind` [18] close the *binary* gap but not the *semantic* gap: the C++ backend cannot reason about high-level mathematical invariants defined in the Python layer.

Recent advances in C++ change this calculus. Three C++23 features are central:

Concepts allow *structural typing*: a type satisfies a concept if it provides the required operations and relations, regardless of its name. This is the compile-time analogue of Haskell type classes, but with zero-cost dispatch and no runtime dictionary.

Non-type template parameters (NTTP) allow lambdas—including predicate functions—to appear as template arguments. A set-builder predicate can therefore be part of the *type*, not merely a runtime value passed at construction.

Named modules enforce a strict directed-acyclic build graph, preventing circular dependencies between algebraic layers and caching verified structural invariants as Binary Module Interface (BMI) files.

Together, these features let the compiler rule out whole classes of algebraic-law violations at translation time and exploit the resulting structural knowledge for optimization. This is *distinct* from theorem-proving: the compiler discharges the proof obligations it is given, it does not search for new ones.

2.2 Structural vs. Nominal Typing: The Juliet Posture

The three schools of type identification — nominal (declared lineage; checked at compile time but blind to behaviour), duck (structural by call-site method probe; checked at runtime, too late for the optimiser), and static structural typing (concept-based; behaviour checked at compile time without a declared lineage) — are textbook material; for the canonical treatment see Pierce [19] and Cardelli's type-systems survey [20].

C++23 concepts realise the third school. We call the resulting discipline the *Juliet Posture*, after the observation that a rose by any other name would smell as sweet [21]: the library cares about the *scent* (structural invariants), not the *lineage* (declared name).

```
// Nominal (pseudocode): recognised by declared lineage.
struct Ring { /* virtual operations, must opt in explicitly */ };
struct MyInt : Ring { /* ... */ };

// Duck typing (pseudocode): recognised at runtime by the call site.
//   def generic_sum(a, b):        # Python: fails only when called
//       return a + b              # no compile-time guarantee

// Juliet Posture (C++23, the actual dedekind concept):
template <typename T>
concept HasRingOperators = requires(T a, T b) {
  { a + b  } -> std::same_as<T>;
  { a - b  } -> std::same_as<T>;
  { -a     } -> std::same_as<T>;
  { a * b  } -> std::same_as<T>;
};
```

```
// int satisfies HasRingOperators without any base class, checked at compile
// time: the operators exist. But the operational concept is deliberately
// weaker than the algebraic laws --- see lst:species --- so a separate
// trait lookup filters out carriers whose + is not actually associative.
static_assert(HasRingOperators<int>);            // operators exist
static_assert(HasRingOperators<Rational<long>>);
static_assert(!is_associative_v<int, std::plus<int>>);  // but + is not a monoid
```

Listing 1: The Juliet Posture: structural recognition checked statically, without requiring declared lineage. The operational concept from `dedekind.algebra:ring`.

The Juliet Posture delivers the best properties of both predecessors: structural sensitivity (behaviour, not name) and static verification (compile time, not runtime). Crucially, because the concept check is a compile-time proof obligation, the compiler can *act on it*: it can specialize code paths, infer additional algebraic properties from the trait registry, and ultimately prune logically empty set expressions to zero machine instructions—none of which is possible with runtime duck typing.

2.3 The Compiler as Structural Gate and Optimization Engine

Two durable uses of a compile-time type system underlie this paradigm, neither of which requires the compiler to be a theorem prover:

1. **Ruling out entire classes of errors.** Encoding algebraic laws as concept requirements makes every instantiation of a generic template an implicit proof obligation discharged at translation time. Failed obligations are type errors, not runtime exceptions. The obligation itself is finite, decidable, and mechanical—the compiler *checks* it, but does not *search for* it.

2. **Enabling entire classes of optimizations.** The LLVM backend exploits the structural knowledge inherited from discharged obligations. When a set is defined by a predicate and two predicates are logically contradictory, the compiler determines at translation time that the resulting intersection is empty and emits no machine instructions for it. This is what we term *structural pruning*, and it is the subject of Section 5.

Both uses are bounded. clang's type checker is not a theorem prover: it has no dependent types, no universal-quantification inference, no proof-search engine. What it does have is `static_assert`, `requires`-clauses, and `constexpr` evaluation—enough to verify a predetermined proof sketch, not enough to generate one. The paradigm claims the reliable half: ruling out, and optimizing under, the classes of structural failures the programmer has named in the type system. The Curry–Howard correspondence between propositions and types is suggestive here, but its full force (dependent products, inductive proofs, universe hierarchies) is not available in C++23; what *is* available is enough to anchor the two uses above.

"Poor man's" is structural, not economic. We call `clang++` a *poor man's* theorem prover throughout the paper; the qualifier is structural, not economic. `clang++` *is* a formal system in the technical sense — a typing derivation it accepts is a proof in the propositional fragment against a fixed inference rule-set — and the architecture imposes a single division of labour: the engineer carries the honesty obligation up front (the trait registry, concept formulations, no `reinterpret_cast` in the public surface), and the compiler discharges every `static_assert` / `requires`-clause / `constexpr` evaluation that follows. The Curry–Howard reading [4] grounds this technically; codified engineering ethics ground it professionally — the NSPE *Code of Ethics for Engineers* [22] canons III + IV (truthfulness in public statements; faithful agency) treat a trait specialisation as a public statement subject to the canons. The carrier-lattice's recurring

Honest Rejection policy is the clearest example: refusing `unsigned int` as a witness for $\mathbb{N}$ is not pedantry but the kind of informal-claim re-examination Canon III asks of a working software engineer. The trade is an honesty obligation up front for a mechanical guarantee afterward.[3]

Bounded yet open-ended. Each use of the technique is bounded individually, but every class of errors ruled out and optimisations enabled tends to expose a next class that the current machinery does not handle but the library's structural vocabulary already names. The four axes of Section 5.6 fall out of the halfspace reductions almost by construction.[4]

Methodological disclosure: AI-assisted development. The library is developed with AI assistance (Anthropic's Claude as the primary code-writing agent, with Gemini and GitHub Copilot in supporting roles); the human role is prioritisation, orchestration, and consensus-before-merge. Three compounding constraints keep the artefacts honest: *domain textbook anchoring* (concept names and law statements taken directly from the published mathematical vocabulary — Lawvere, Mac Lane, Lambek, Wadler, Moggi), *mechanical validation* (every algebraic claim becomes a `static_assert` the compiler discharges; every concept composes through a named-module DAG; every behavioural claim passes CI), and *codomain textbook anchoring* (concepts validated against the C++ standard library's own ontology — `std::ranges::input_range`, `std::regular`, `std::numeric_limits`, the iterator hierarchy). The natural-language fragment of the workflow is a division of labour: the mathematical content stays in mathematical vocabulary all the way down to where the machine takes over.

3 Intensional First, Realize When You Mean It

3.1 The Design Rule

The central methodology of `dedekind` is captured by a single rule:

> *Intensional first, realize it when you mean it.*

The thrust of this rule is to leverage *lazy evaluation*—both at the type level (compile-time symbolic substitution) and at the value level (deferred numeric approximation)—as a principled defence against *Premature Materialization*: the loss of mathematical structure that occurs when a computation is forced to a concrete representation earlier than necessary.

An *intensional* description specifies a mathematical object by its defining properties. An *extensional* representation enumerates its members or computes its value. A halfspace $\{x \in \mathbb{Q} \mid x \leq 4\}$ is intensional; the list of its rational points between 0 and 4 is extensional; and a function that materialises the former into the latter is the auditable crossing of that boundary. The Dedekind cuts of Section 3.3—where a real number is named by the pair of rational sets below and above it—push the same distinction to its limit: the real is intensional forever, and only bounded-precision queries cross the boundary. This framing is close in spirit to intensional database semantics (for example, Majkić's intensional FOL treatment for peer-to-peer data integration [25]), but our engineering contrast with mainstream SQL is explicit: SQL evaluation is typically defined over already-realized relations, while `dedekind` preserves predicate-level structure in the type system and defers materialization to an explicit realization boundary.

[3] The argument is bounded. Most software claims (TCP retransmit-timer correctness, scheduler fairness, end-to-end latency) cannot be reduced to type-level structural assertions. The paper covers the fragment where the reduction is available; Section 5.6 names the bounds.

[4] Penrose's *The Road to Reality* [23] offers an analogous observation about Gödel's incompleteness theorem — a formal system closing at one level motivates the next, not as exhaustion but as natural successor; the closest formal analogue is Turing's ordinal logics [24], where each stage's consistency feeds the next stage's formation rule. The systems-programming reading is benign: the issue tracker plays the successor role.

The methodological lineage also runs through Abelson and Sussman's [26] *programming by wishful thinking*: name the procedure you wish you had, use it, and supply its definition only when the surrounding code forces a concrete answer. SICP's medium for this discipline is Scheme—an untyped lambda calculus where the wished-for procedure is *notational* until evaluation reaches it. Our medium is the C++23 type checker, which performs a typed lambda calculus on our behalf at compile time: the wished-for object inhabits a predicate-set type, the wishes are tracked structurally as type-level constraints, and the realization boundary is the single point at which the type system permits an extensional value to be produced. Where SICP defers *evaluation* (a runtime move), we defer *materialization* (a compile-time-to-runtime move), and the type checker is the auditor that ensures the wishes were coherent before the realization happens.

$$
\begin{tikzcd}
\texttt{Cardinality}\ (\mathbb{N}) \arrow[r, "\texttt{lift\_}\mathbb{N}\texttt{\_}\mathbb{Z}\texttt{\_}"] & \texttt{SignedCardinality}\ (\mathbb{Z}) \arrow[l, dashed, bend right, "\texttt{abs}"'] \\
\texttt{unsigned int}\ (\mathbb{Z}/2^w\mathbb{Z}) \arrow[u, "\texttt{embed\_uint\_}\mathbb{N}\texttt{\_}"] \arrow[r, "\texttt{embed\_uint\_sint\_}"] & \texttt{int}\ (\text{machine } \mathbb{Z}) \arrow[u, "\texttt{embed\_sint\_}\mathbb{Z}\texttt{\_}"'] \\
\texttt{bool}\ (\mathbb{B}) \arrow[u, "\texttt{embed\_}\mathbb{B}\texttt{\_uint\_}"] \arrow[r, "\texttt{embed\_}\mathbb{B}\texttt{\_}\mathbb{K}\texttt{3\_}"] & \texttt{Ternary}\ (\mathbb{K}3) \arrow[uu, bend right=60, "\texttt{embed\_}\mathbb{K}\texttt{3\_}\mathbb{Z}\texttt{\_}"']
\end{tikzcd}
$$

Figure 1: The carrier lattice (discrete-foundations slice). Three layers, bottom to top: truth-value carriers (`bool`, `Ternary`); machine integer carriers (`unsigned int` $= \mathbb{Z}/2^w\mathbb{Z}$ as a finite cyclic ring; `int` = machine $\mathbb{Z}$ with UB-on-overflow); variant carriers inhabiting the textbook Forms (`Cardinality` $= \mathbb{N}$ as the Natural Numbers Object; `SignedCardinality` $= \mathbb{Z}$ as the Initial Ring / Grothendieck group of $\mathbb{N}$). Each labelled arrow is an exported `arrow` object in `dedekind.numbers` registered as a monic morphism (the trait `is_monic_arrow_v` = `true`); test case `[carrier-lattice][figure][monic][witness]` pins the seven `IsMonicArrow` concept-level facts that this caption asserts. The top-row variant-layer arrow `lift_`$\mathbb{N}$`_`$\mathbb{Z}$`_` (which wraps the public function `dedekind::sets::lift_cardinality_to_signed`) is the canonical $\mathbb{N} \hookrightarrow \mathbb{Z}$ embedding and is the Grothendieck-construction *unit* at object $\mathbb{N}$; the binary closure-forcing operator `Cardinality` − `Cardinality` → `SignedCardinality` realises the Grothendieck *construction* (the multiplication of the construction) on operator pairs and is not shown on the diagram (it would require a product node). The middle-row arrow `embed_uint_sint_` is the machine-layer sign reinterpretation `static_cast<int>(unsigned)`; renamed (post-#457) to mark the actual machine-layer domain and codomain explicitly — the variant-layer canonical $\mathbb{N} \hookrightarrow \mathbb{Z}$ embedding lives at the top row as `lift_`$\mathbb{N}$`_`$\mathbb{Z}$`_`. The dashed retraction `abs` is the split-mono partner of `lift_`$\mathbb{N}$`_`$\mathbb{Z}$`_` (`abs` ∘ `lift` = id on the non-negative fragment); it is not asserted monic — `abs` folds sign (`abs`(3) = `abs`(-3)), so distinct $\mathbb{Z}$-values are conflated. `embed_`$\mathbb{K}$`3_`$\mathbb{Z}$`_` skips the machine-int row and lands directly on `SignedCardinality` (a deliberate factorisation choice). Multiple paths between extremes commute up to canonical isomorphism.

Forms before carriers. The library reifies the discrete-foundations triple structurally rather than starting from a machine-integer carrier: $\mathbb{B}$ as the Subobject Classifier Ω (ETCS Axiom 7), $\mathbb{N}$ as Lawvere's Natural Numbers Object (NNO; ETCS Axiom 9 [27]), and $\mathbb{Z}$ via two universal properties on a single carrier — initial object in **CRing** and free abelian group on the commutative monoid $(\mathbb{N}, +, 0)$ (the Grothendieck construction; see Mac Lane [28] for the standard

$$
\begin{tikzcd}
 & \texttt{SignedCardinality}\ (\mathbb{Z}) \arrow[dl, "\chi_{\mathbb{Z}/2}"'] \arrow[d, "\chi_{\mathbb{Z}/5}"] \arrow[dr, "\chi_{\mathbb{Z}/7}"] & \\
\texttt{Modular}\langle 2\rangle & \texttt{Modular}\langle 5\rangle & \texttt{Modular}\langle 7\rangle
\end{tikzcd}
$$

Figure 2: The universal-property reading of $\mathbb{Z}$ as the initial ring (sibling to Figure 1). For every commutative unital ring R, the initial-object property of $\mathbb{Z}$ supplies a unique ring homomorphism $\chi_R : \mathbb{Z} \to R$. The diagram exhibits this at three concrete prime targets $R = \texttt{Modular}\langle n\rangle$ (the cyclic ring $\mathbb{Z}/n\mathbb{Z}$, with prime $n \in \{2, 5, 7\}$ chosen so each target is a field). Each χ_n is the mod-n reduction; the test suite `[algebra][initial_ring][modular][universal-property]` (in `src/test/cpp/modules/dedekind/numbers/initial_ring_test.cpp`) realises χ_n operationally as the helper `χ_to_modular<n>` and exercises ring-homomorphism preservation ($\chi(a + b) = \chi(a) + \chi(b)$, $\chi(a \cdot b) = \chi(a) \cdot \chi(b)$, $\chi(1_\mathbb{Z}) = 1_R$) plus sign-aware mod reduction. A complementary structural reading: `unsigned int` $\cong$ `Modular`$\langle 2^w\rangle$ at the machine layer (the carrier-lattice middle row of Figure 1 is the same diagram unrolled at the machine width); `bool` $\cong$ `Modular`$\langle 2\rangle$ under XOR. The figure is the visual companion to the engineer's honesty obligation that universal-property uniqueness imposes: declaring `is_initial_ring_v<SignedCardinality> = true` is the type-system anchor; the operational behaviour at every concrete target is the test-suite obligation.

treatment).[5]

3.2 The Tension: Abstract Naturals vs. IEEE Numerics

Here is the tension in one sentence: the `int` in your source file is not the $\mathbb{Z}$ in your textbook, and everybody quietly agrees to pretend it is. The ring $\mathbb{Z}$ obeys a few laws:

- Associativity of addition: $(a + b) + c = a + (b + c)$.
- Existence of additive inverse: $\forall a, \exists -a$ such that $a + (-a) = 0$.
- No upper or lower bound.

A 32-bit signed `int` obeys *none* of them exactly: addition overflows, negation of `INT_MIN` is undefined behaviour, and the representable range is finite. IEEE 754 `double` [6] is further out still: addition is non-associative because rounding happens every step, and `NaN` and $\pm\infty$ puncture the law of excluded middle in the exact way Section 4 will ask the subobject classifier Ω to repair. A generic algorithm that silently assumes the ring laws on these carriers is, in the literal technical sense, wrong—it just happens not to be wrong on most inputs.

`dedekind` encodes these divergences explicitly in the `:species` trait registry:

```
// Variable template: is_associative_v<T, Op> is the structural trait.
// The library partial-specialises it per-carrier, per-operation.

// Unsigned integrals are associative under +  (modular arithmetic).
template <std::unsigned_integral T>
inline constexpr bool is_associative_v<T, std::plus<T>> = true;
```

[5] Each Form sits behind a three-layer chain *Form* $\to$ *Carrier* $\to$ *Symbol*: a universal-property concept (`IsNNO`, `IsInitialRing`, `IsGrothendieckGroup`), a canonical inhabitant (`Cardinality` for $\mathbb{N}$ with $\aleph_0$ saturation; `SignedCardinality` for $\mathbb{Z}$ with $\pm\aleph_0$ and `NaZ`), and the practitioner-facing alias. The negafibonacci recurrence $F_{n-1} = F_{n+1} - F_n$ is the canonical exhibit: the backward step requires exactly the operator (subtraction) that distinguishes $\mathbb{Z}$ from $\mathbb{N}$, and the variant carriers escalate to $\pm\aleph_0$ rather than overflowing as `int`/`unsigned int` would.

```
// Signed integrals are REJECTED: overflow of a signed integer is
// undefined behaviour, so the addition operator is not a law-abiding
// monoid on signed carriers.
template <std::signed_integral T>
inline constexpr bool is_associative_v<T, std::plus<T>> = false;

// IEEE double is rejected for a different reason: rounding breaks
// bit-exact associativity.
template <>
inline constexpr bool is_associative_v<double, std::plus<double>> = false;

static_assert(is_associative_v<unsigned int, std::plus<unsigned int>>);
static_assert(!is_associative_v<int,          std::plus<int>>);
static_assert(!is_associative_v<double,       std::plus<double>>);
```

Listing 2: The species registry records algebraic properties—and their absence. Signed `int` is explicitly rejected as associative under `+` because signed overflow is undefined behaviour.

Generic algorithms that require associativity will fail to compile for `int` or `double` alike, unless an explicit numeric policy (modular semantics on signed overflow, compensated summation, an interval-arithmetic wrapper, or a lift to `Rational<long>`) is supplied. The realization boundary is the point at which the programmer chooses which policy to apply.

3.3 Dedekind Cuts as the Canonical Example

Dedekind's construction — $r \in \mathbb{R}$ as the partition (L, U) of $\mathbb{Q}$ into rationals strictly below r and those at or above — translates the intensional-first rule directly: the two sets are type-level predicates, and *the predicate is the number*. The library's canonical worked example is Dedekind's own $\sqrt{2}$: a rational q is in the lower cut iff $q^2 < 2$, a decidable predicate over $\mathbb{Q}$ that compiles into a finite type-level object — no approximation, no series summation:

```
template <typename Q>
constexpr auto Sqrt2_Symbolic() {
  const dedekind::sets::Ω<Q, TernaryLogic> universe{};
  // The lower cut L_sqrt2 = { q in Q | q^2 < 2 }.
  return ambient_set<Q>([universe](const Q& q) {
    if constexpr (std::floating_point<Q>) {
      if (std::isnan(q)) { return Ternary::Unknown; }
    }
    const auto in_cut =
        (q * q < static_cast<Q>(2)) ? Ternary::True : Ternary::False;
    return TernaryLogic::AND(universe(q), in_cut);
  });
}
```

Listing 3: $\sqrt{2}$ as an intensional lower cut over $\mathbb{Q}$, from `dedekind.numbers:symbolic`. The predicate is the number.

No floating-point approximation is computed; no Newton iteration runs. Asking "is $\frac{7}{5}$ less than $\sqrt{2}$?" reduces to $\frac{49}{25} < 2$, decidable at translation time. Approximation to a concrete float is available later, at a specific precision, when the programmer asks for it. The intensional object survives indefinitely without losing reasoning power.

3.4 Carriers: From Exact Rationals to Packed Integers

The intensional-first rule applied to *every* numeric role at once is spelled out in three tables. Table 1 catalogs the *exact* side: the library's own intensional carriers for each mathematical role, and the algebraic concept each realises. Table 2 catalogs the *primitive* side: the `std`-library

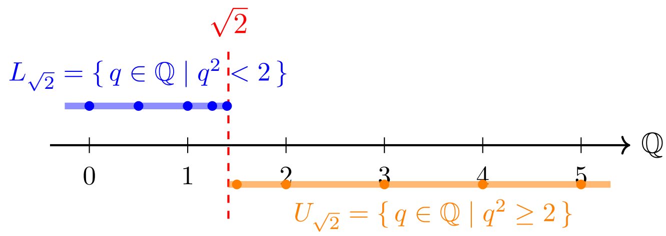

Figure 3: A Dedekind cut names $\sqrt{2}$ by partitioning $\mathbb{Q}$. The lower half $L_{\sqrt{2}}$ (blue) and upper half $U_{\sqrt{2}}$ (orange) are both compile-time predicates. In `dedekind`, the real number *is* the partition: no approximation is stored, no square root is computed, and the compiler still knows everything about $\sqrt{2}$ that the predicate $q^2 < 2$ expresses.

types the hardware runs natively, and the concept each of them carries (with the usual IEEE / overflow caveats). Table 3 is the state-machine that connects the two: given a role, which named embedding or realisation arrow moves you from exact to primitive, and which moves you back.

Table 1: **Exact carriers.** The library's intensional types, one per mathematical role, together with the algebraic concept each satisfies. These are the carriers a disciplined engineer writes the *reference implementation* against — laws are provably satisfied, not approximated.

Role	Exact carrier	Concept satisfied
$\mathbb{B}$	`bool` under $\wedge, \vee$	`IsBoolean`
$\mathbb{F}_2$	`bool` under $(\oplus, \wedge)$	`IsGaloisField<bool,^,&>` (order 2)
$\mathbb{F}_{64}$	`𝔽64` $(\mathbb{F}_2[x]/(x^6+x+1)$, 6-bit in `uint8_t`)	`IsGaloisField<T,+,*>` (order 64)
$\mathbb{F}_2^{64}$	`uint64_t` under $\oplus$, scalar action `Bit2U64Action`	`IsVectorSpace<uint64_t, bool>`
$\mathbb{N}$ bounded	`ExtensionalCardinal<N>`	`IsCyclicGroup`, `IsAbelianGroup<T,+>`
$\mathbb{N}$ unbounded	`Cardinality` (variant over $\aleph_0$)	saturating commutative monoid via `add`/`mul` helpers
$\mathbb{Z}$ bounded	`SignedExtensionalCardinal<N>`	`IsAbelianGroup<T,+>`, `IsRing`
$\mathbb{Z}$ unbounded	`SignedCardinality` (variant over $\pm\aleph_0$, NaZ)	`IsAbelianGroup<T,+>`, `IsRing`
$\mathbb{Q}$	`Rational<SignedExtensionalCardinal<>>`	`Field_ℚ`
$\mathbb{R}$ symbolic	`LowerCut<Rational>` (Dedekind cut)	`Continuum_ℝ` (predicate-only)
$\mathbb{C}$	`Complex<Rational<SignedEC<>>>`	`Algebra_ℂ`
$\mathbb{D}$ (dual)	`Dual<Rational<SignedEC<>>>`	`IsRing<T,+,*>`, $\varepsilon^2 = 0$

A fourth table addresses where textbook ring structure and literal C++ operators diverge: the library expresses the disagreement through three concept tiers — *strict* (categorical proof), *functor-parametric shape* (operators close on the chosen functors), and *literal shape* (the C++ `+`, `-`, `*` close on the carrier directly) — sealed by `IsArithmeticRing` when all three agree under `(std::plus, std::multiplies)`. Table 4 catalogues the divergences on the carriers used in the worked examples (Issue #393); the friction list is summarised structurally in Section 5.8.

Reading rule for callsites: pick *seal* for plain C++ syntax over a textbook-aligned carrier; pick *strict + functor* when the carrier carries its ring structure over non-standard functors (`bool` as $\mathbb{F}_2$, narrow unsigneds with `std::plus` / `std::multiplies`); pick *literal* for the operator surface without the algebraic axioms (elementwise-lifted matrix bodies, halfspace evaluation).

Migration discipline. The three tables form a migration map: *start correct* on Table 1 (typically `Rational<SignedExtensionalCardinal<>>` for arithmetic-heavy code, with bit-for-bit provable laws); *cross over* via Table 3 only in profiled inner loops where arithmetic cost dominates; *land* on Table 2 with the named caveat the concept cell records (e.g. loss of associativity under `+`, which some algorithms tolerate and others do not). The rule: **start correct, finish**

Table 2: **Primitive carriers.** The `std`-library types the hardware runs natively, with the algebraic concept each carries under its C++-standard arithmetic semantics. These are the carriers a disciplined engineer drops to for performance, *after* verifying the reference implementation on the exact side.

Primitive type	**Arithmetic semantics**	**Concept satisfied**
`bool`	Boolean algebra under $\wedge, \vee, \neg$	`IsBoolean`
`unsigned long`, `size_t`	Modular, wraps at 2^N	`IsCyclicGroup`, `IsAbelianGroup<T,+>`
`int`, `long`	Partial (UB on signed overflow)	`HasRingOperators`[a]
`int8_t`, `int16_t`	Partial + SIMD-vectorisable	`HasRingOperators`[a]
`float`, `double`	IEEE 754, rounded, non-associative	`Continuum_`$\mathbb{R}$ (non-assoc. `+`)
`std::complex<double>`	Cartesian pair over IEEE `double`	`Algebra_`$\mathbb{C}$ (non-assoc.)

[a] `HasRingOperators` holds (literal $+$, binary $-$, unary $-$, $*$ all close on the carrier); the strict `IsRing` bundle refuses because signed-overflow UB makes the operation partial, which doesn't satisfy any of the `IsTotal` paths (`IsPeriodic` / `IsIdempotent` / `IsSaturating`) the totality gate admits (math-wins-over-C++). Total alternatives: `SignedCardinality` (saturating), `SignedExtensionalCardinal<>` (cyclic), or the `:partial` Maybe-monad chain for honest fail-on-overflow.

Table 3: **Crossings state-machine.** The named morphisms that move between exact and primitive sides, per role. Each row shows *realise* (exact $\to$ primitive: drop exactness for speed) and *embed* (primitive $\to$ exact: lift a machine value into the certified carrier). A dash marks a direction that requires going through an intermediate role.

Role	**Realise: exact $\to$ primitive**	**Embed: primitive $\to$ exact**
$\mathbb{B}$	(identity)	(identity)
$\mathbb{N}$	`.realize_to_size_t(sentinel)`	`embed_unsigned_integral<N>(v)`
$\mathbb{Z}$	`static_cast<int64_t>(x)`	`embed_signed_integral<Z>(v)`
$\mathbb{Q}$	`.resolve()` $\to$ `double`	`Rational<Z>(num, den)`
$\mathbb{R}$ symbolic	evaluate cut at a rational bound	(intensional; no simple lift)
$\mathbb{R}$ policy	unwrap `IEEE<double>`	wrap in `IEEE<double>`
$\mathbb{C}$	per-component `.resolve()`	`Complex<R>(re, im)`
$\mathbb{D}$ (dual)	`.val` (primal only; tangent discarded)	`Dual<F>(v, d)`

Table 4: **Math textbook ↔ C++ standard.** Where the literal C++ operators agree with the textbook (rows that tick all columns), the bundled seal `IsArithmeticRing` fires and generic code can be written in plain syntax. Where they diverge (integer promotion, signed-overflow UB), the table names the precise failure point and the tier that still fires. "∘" marks an assertion that fires only at the literal-shape tier (the strict half is gated by an open FIXME on the species-trait registry); "–" marks not-currently-asserted.

Carrier	**Math claim**	**Strict**	**Functor**	**Literal**	**Seal**
`unsigned int`, `unsigned long`, `size_t`	ring $\mathbb{Z}/2^N\mathbb{Z}$	✓	✓	✓	✓
`unsigned short`, `unsigned char`	ring $\mathbb{Z}/2^N\mathbb{Z}$	✓	✓	×[a]	×
`int`, `long`	partial (signed-overflow UB)	×[b]	✓	✓	×
`bool` under $(\oplus, \wedge)$	Galois field $\mathbb{F}_2$	✓	✓	×[a]	×[c]
`Rational<long>`	field $\mathbb{Q}$	∘[d]	∘	✓	∘
`RigPolynomial<unsigned int>`	polynomial ring $R[x]$	✓	✓	✓	✓
`Matrix2x2V<unsigned int>`	non-commutative ring	∘	∘	✓	–
`Vec2V<T>`	module / vector space, *not a ring*	n/a	n/a	×[e]	n/a

[a] integer promotion lifts the result of literal `+` / `*` to `int`; the functor surface returns `T` by signature.
[b] signed-overflow UB means the carrier admits no `IsTotal` label (it is neither periodic, nor idempotent, nor saturating); the categorical `IsRing` chain — which composes `IsTotal` — therefore breaks for this carrier.
[c] `bool`'s ring structure lives over $(\oplus, \wedge)$, not $(+, *)$; `IsArithmeticRing`, which fixes `(std::plus, std::multiplies)`, asks the wrong question for this carrier.
[d] operational under the active numeric policy: `IsFieldLikeScalar` is the operational variant, the categorical proof is intentionally not specialised.
[e] vectors have no internal product `Vec2V $\times$ Vec2V $\to$ Vec2V`, only scalar multiplication; the shape concept correctly refuses.

fast, and the type system enforces that the laws your code relies on are the laws the carrier actually provides — or fails to compile.

4 Sets Are Rules, Not Buckets

4.1 The Comprehension View

Here is the move. A set is not a bucket. A set is a rule:

$$S = \{x \in A \mid P(x)\}.$$

The right-hand side is the definition most mathematicians write down first and then silently forget, reaching instead for a list or a `std::vector`. In `dedekind` we take that right-hand side literally. The set S is the *type* whose inhabitants are elements of the ambient species A satisfying P. Nothing is enumerated, nothing is allocated, nothing is stored. The predicate *is* the set.

That shift—what Lawvere called the comprehension view [2, 29]—is the one idea this section needs. Once you accept it, everything downstream (intersection, complement, emptiness, pruning) falls out as predicate algebra. The subobject classifier Ω becomes a knob you can turn: classical `bool` for exact domains, Kleene K3 for floats, anything else you can register.

```
using namespace dedekind::category;

// ambient_set<T>(P) creates the intensional set {x in T | P(x)}.
// No values are stored; the predicate IS the set.
constexpr auto even_gt_ten = ambient_set<int>(
    [](const int& x) { return x % 2 == 0 && x > 10; });

// Membership is a function call returning a logic value; no runtime
```

```
// container or heap allocation is involved.
static_assert(even_gt_ten(12) == ClassicalLogic::True);
static_assert(even_gt_ten(7)  == ClassicalLogic::False);
```

Listing 4: A set as a compile-time predicate type. The set object is invocable; membership returns a logic value.

4.2 Set Operations as Predicate Composition

If a set is a predicate, a set operation is whatever operation you would apply to the predicates. The dictionary writes itself:

$$\begin{aligned} S_1 \cap S_2 &\triangleq \{x \in A \mid P_1(x) \wedge P_2(x)\} \\ S_1 \cup S_2 &\triangleq \{x \in A \mid P_1(x) \vee P_2(x)\} \\ \overline{S_1} &\triangleq \{x \in A \mid \neg P_1(x)\} \end{aligned}$$

Every operation on the left is an operation on Ω on the right. Intersection is conjunction; union is disjunction; complement is negation. No values are touched, because there are no values yet. The combinators compose at compile time and produce a new type—a new rule—which the compiler is then free to inspect.

```
using namespace dedekind::category;

// Two intensional sets over int.
constexpr auto evens    = ambient_set<int>(
    [](const int& n) { return n % 2 == 0; });
constexpr auto ten_plus = ambient_set<int>(
    [](const int& n) { return n > 10; });

// Intersection: the predicate of (evens & ten_plus) is the conjunction
// of the two component predicates, built as a type by the compiler.
constexpr auto even_and_ten_plus = evens & ten_plus;

static_assert(even_and_ten_plus(12) == ClassicalLogic::True);
static_assert(even_and_ten_plus(11) == ClassicalLogic::False);
static_assert(even_and_ten_plus(4)  == ClassicalLogic::False);
```

Listing 5: Set intersection as type-level predicate conjunction, discharged by the library's `operator&` on `Set`.

4.3 The Subobject Classifier Ω

Here is what Ω is doing for us. A predicate $P : A \to \Omega$ maps ambient elements to *truth values*, and Ω is the type those truth values live in. Change Ω, and you change what counts as membership.

The default is the one every programmer expects: $\Omega = \{\top, \bot\}$, classical two-valued logic. Plug it in and you get ordinary set theory.

Swap it for Kleene K3—$\Omega = \{-1, 0, +1\}$—and something more interesting happens. The extra value is *Unknown*, and it is where `NaN` lives. In classical logic, $P \vee \neg P = \top$ is a theorem; with `NaN` in play, that theorem quietly breaks, because neither $x <$ `NaN` nor $x \geq$ `NaN` is true. In K3, $P \vee \neg P = \mathit{Unknown}$ is a calm, unremarkable identity. The paradox was never in the floats; it was in our choice of Ω.

That is the payoff: the same comprehension machinery handles exact domains (pick classical Ω) and floating-point domains (pick K3) with no change to the API. The logic is a parameter.

5 Compile-Time Reductions: From Set Pruning to Optimisation and Calculus

5.1 Structural Pruning in Practice

The payoff of the comprehension view is that the LLVM backend can *prune* a set expression to a no-op when its predicate is statically unsatisfiable.

```
using namespace dedekind::order;

constexpr auto n = var<>;

// Two halfspace-structured sets, with pivots (5 and 3) carried as NTTPs.
constexpr auto gt_five  = Set{n % N | (n > bound<5>)};
constexpr auto lt_three = Set{n % N | (n < bound<3>)};

// operator& dispatches through `structured_and`; the type-level
// contradiction rule collapses the meet to the empty-set type ∅<int>.
constexpr ∅<int> empty_meet = gt_five & lt_three;
static_assert(empty_meet == ∅{});
```

Listing 6: Contradictory predicates collapse to the empty-set type $\varnothing$. Uses the `Halfspace`-NTTP DSL to pin both bounds in the type signature, so the contradiction is resolved at translation time.

The generated binary contains no membership-test code for `empty_meet`: the type-level contradiction rule resolves the result to $\varnothing$ *before* code generation, and the computability tier of $\varnothing$ is fully top (extensional/classical/element-in-type). The showcase listings below make this mechanically observable.

Showcase witnesses

Two concrete witnesses from the test suite demonstrate this in the actual `dedekind` API.

Showcase 1 — Diagonal contradiction in $\mathbb{R}^2$. On the diagonal $\{(x, y) \mid x = y\}$, the strip predicate $x > 5 \wedge y < 3$ is contradictory (since $x = y > 5$ forces $y > 5$, violating $y < 3$). The intersection is proven empty by `static_assert`; the exported function compiles to a single `ret i1 false`.

```
constexpr auto R2 = R * R;
constexpr auto xy = var<decltype(R2)>;
using R2Point = typename decltype(R2)::Domain;

// Diagonal: { (x, y) ∈ ℝ² | x = y }
constexpr auto diagonal =
    Set{xy % R2 | [](R2Point p) { return p.first == p.second; }};

// Strip: { (x, y) ∈ ℝ² | x > 5 ∧ y < 3 }
constexpr auto strip = Set{
    xy % R2 | [](R2Point p) { return (p.first > 5.0) && (p.second < 3.0); }};

constexpr auto empty_diagonal_cut = diagonal & strip;
using R2Logic = typename decltype(empty_diagonal_cut)::logic_species;

static_assert(empty_diagonal_cut(R2Point{6.0, 6.0}) == R2Logic::False);

extern "C" __attribute__((noinline)) bool witness_empty_diagonal_cut() {
  return empty_diagonal_cut(R2Point{6.0, 6.0}) == R2Logic::True;
```

```
}
```

Listing 7: Showcase 1 source: diagonal contradiction in $\mathbb{R}^2$.

```
define noundef zeroext i1 @witness_empty_diagonal_cut() local_unnamed_addr #0 {
  ret i1 false
}
```

Listing 8: Showcase 1 LLVM IR (`clang++-22 -O2`): constant false return.

Showcase 2 — Lattice/square singleton in $\mathbb{C}$. The natural-number lattice (Gaussian integers, $0 \leq \mathrm{Re}, \mathrm{Im} \leq 3$) intersected with the square $[0.5, 1.5]^2$ contains exactly $c_3 = 1 + i$. The compiler proves all four membership assertions at translation time and folds the exported function to a single `ret i1 true`.

```
constexpr auto c = var <>;

constexpr auto natural_lattice_in_c =
    Set{c % C | [](const Complex<double>& z) {
      return is_integral_coordinate(z.real()) &&
             is_integral_coordinate(z.imag()) &&
             (z.real() >= 0.0) && (z.real() <= 3.0) &&
             (z.imag() >= 0.0) && (z.imag() <= 3.0);
    }};
constexpr auto square_c1_c2 = Set{c % C | [](const Complex<double>& z) {
  return (z.real() >= 0.5) && (z.real() <= 1.5) &&
         (z.imag() >= 0.5) && (z.imag() <= 1.5);
}};
constexpr auto lattice_square_intersection = natural_lattice_in_c & square_c1_c2;
using CLogic = typename decltype(lattice_square_intersection)::logic_species;

static_assert(lattice_square_intersection(Complex<double>{1.0, 1.0}) == CLogic::True);
static_assert(lattice_square_intersection(Complex<double>{0.0, 1.0}) == CLogic::False);

extern "C" __attribute__((noinline)) bool witness_lattice_square_singleton() { /* ... */
```

Listing 9: Showcase 2 source: lattice/square singleton in $\mathbb{C}$.

```
define noundef zeroext i1 @witness_lattice_square_singleton() local_unnamed_addr #0 {
  ret i1 true
}
```

Listing 10: Showcase 2 LLVM IR (`clang++-22 -O2`): constant true return.

Showcases 1 and 2 are checked in as LLVM IR fixtures and validated in CI (`make ir-fixture-check`). They exemplify the base case: pair-level predicate composition where constant-folding does the work.

From lambdas to structured predicates: the halfspace DSL

Lambda-based predicates let the optimiser fold at specific call sites, but they erase the *structure* of the constraint. A richer pattern emerges when halfspace bounds live in the *type*: $\{n \in \mathbb{N} \mid n > 5\}$ is represented as `Halfspace<int, 5, Up, Strict>`, with the pivot as a non-type template parameter. Contradiction and cardinality analysis then run in the type system itself, and the reductions are *structural*: no appeal to runtime values is needed.

Showcase 3 — Halfspace contradiction on $\mathbb{N}$. Two opposing halfspaces with non-bridging pivots; the meet reduces to $\varnothing$ by the type-level contradiction rule.

```
constexpr auto n = var <>;
```

```
constexpr auto gt_five  = Set{n % N | (n > bound<5>)};
constexpr auto lt_three = Set{n % N | (n < bound<3>)};

// Set-level `&` dispatches through `structured_and` on the halfspace types;
// the contradiction collapses the result's type to ∅<int>.
constexpr ∅<int> empty_meet = gt_five & lt_three;
static_assert(empty_meet == ∅{});

// Computability as a compile-time observable: the parent Sets carry NONE of
// the three tiers; the reduced ∅ carries ALL THREE.
static_assert(!HasDecidableMembership<decltype(gt_five)>);
static_assert(!IsFiniteSet<decltype(gt_five)>);
static_assert(!IsCompileTimeEnumerable<decltype(gt_five)>);
static_assert( HasDecidableMembership<decltype(empty_meet)>);
static_assert( IsFiniteSet<decltype(empty_meet)>);
static_assert( IsCompileTimeEnumerable<decltype(empty_meet)>);
```

Listing 11: Showcase 3 source: halfspace contradiction on $\mathbb{N}$ collapses to $\varnothing$.

```
define noundef zeroext i1 @witness_empty_halfspace_meet() local_unnamed_addr #0 {
  ret i1 false
}
```

Listing 12: Showcase 3 LLVM IR: constant false return.

Showcase 4 — Cardinality-1 reduction on $\mathbb{N}$. The same meet operator, applied to two halfspaces whose bounds pin exactly one integer: the result's type *is* `Singleton<4>`, with the unique inhabitant in the template parameter.

```
constexpr auto n = var <>;

constexpr auto gt_three = Set{n % N | (n > bound<3>)};
constexpr auto lt_five  = Set{n % N | (n < bound<5>)};

// On an integral carrier the meet's cardinality is compile-time-decidable.
// Cardinality 1 elevates the reduction from OrderInterval to Singleton.
constexpr Singleton<4> in_between = gt_three & lt_five;
static_assert(in_between == Singleton<4>{});

// Same computability tightening as Showcase 3, with the element now in the type.
static_assert(!IsCompileTimeEnumerable<decltype(gt_three)>);
static_assert( IsCompileTimeEnumerable<decltype(in_between)>);
```

Listing 13: Showcase 4 source: cardinality-1 meet collapses to $\{4\}$.

```
define noundef zeroext i1 @witness_halfspace_singleton() local_unnamed_addr #0 {
  ret i1 true
}
```

Listing 14: Showcase 4 LLVM IR: constant true return at the unique inhabitant.

The two halfspace showcases make the central claim of the paper mechanically observable: compile-time reduction of an intensional description over a transfinite carrier to a named extensional object (either $\varnothing$ or a `Singleton`) monotonically restores decidability, finiteness, and type-level knowledge of elements. The *reduction boundary* is the *computability boundary*, and the six `static_asserts` flanking each reduction are its mechanical witness.

FIXME (#362) — showcase: carrier strength-reduction. The showcases above collapse the *cardinality* of an intensional set (transfinite $\mapsto$ finite $\mapsto$ singleton); a complementary showcase, not yet listed in the showcase grid, would collapse the *ambient set* itself. Mathematically: $A \subset \mathbb{N}$, $B \subset \mathbb{Z}$, $\mathbb{N} \hookrightarrow \mathbb{Z}$ implies $A \cap B \subset \mathbb{N}$ (not $\subset \mathbb{Z}$), and dually $A \cup B \subset \mathbb{Z}$ widens along the embedding. The library now ships an existential proof of the rule on the variant pair $(\texttt{Cardinality}, \texttt{SignedCardinality})$ — $\texttt{Set}\langle\mathbb{N}\rangle \cap \texttt{Set}\langle\mathbb{Z}\rangle$ type-collapses to $\texttt{Set}\langle\mathbb{N}\rangle$, locked in by unit tests — but the mechanism does not yet appear as an IR-fixture showcase with a `ret i1 true / false` witness on a heterogeneous intersection. Adding such a showcase makes the carrier-axis collapse empirically observable on the same footing as the cardinality-axis ones above; it sits naturally next to Showcases 3 and 4 in the grid. The full carrier-promotion framework (any pair in the lattice $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R} \subset \mathbb{C}$, not just the variant pair) is tracked under issue #362 and discussed in `docs/design/carrier-lattice.md`.

Further showcases in the source tree exercise the same machinery on $\mathbb{R}$ ambient carriers (showcase 5), non-point intervals with observable cardinality (showcase 6: $-21 < n \leq 21$ on $\mathbb{Z}$, $|\cdot| = 42$), integer lattice intersected with real-valued bounds (showcase 7), and 2D rectangles via structural cartesian product (showcase 8: $42 \times 11 = 462$). All are IR-verified.

The machine layer below $\mathbb{N}$: `std::unsigned_integral` is $\mathbb{Z}/2^w\mathbb{Z}$. The lattice $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R} \subset \mathbb{C}$ rests on a machine layer occupied by the standard library's primitive integer families. The honest textbook reading of `std::unsigned_integral` is that each width-w instance is the finite cyclic ring $\mathbb{Z}/2^w\mathbb{Z}$ under modular arithmetic — a *commutative ring*, *not* the textbook $\mathbb{N}$.[6] The library's $\mathbb{N}$ lives on the variant carrier `Cardinality` (the saturating commutative monoid, no $-$); the `std::unsigned_integral` layer is the canonical quotient of monoids $\mathbb{N} \twoheadrightarrow \mathbb{Z}/2^w\mathbb{Z}$ that sits *above* $\mathbb{N}$ rather than coinciding with it.[7] The truthy-positivity injection $\mathbb{B} \hookrightarrow$ `std::unsigned_integral` and the structure-forgetting lift `std::unsigned_integral` $\to$ `Cardinality` make the embedding chain $\mathbb{B} \hookrightarrow$ `std::unsigned_integral` $\to \mathbb{N}$ explicit at the type level. The full discussion lives in the report chapter on the carrier lattice.

The signed sibling: `std::signed_integral` carries the operator surface but not the axiomatic ring. The signed-integer family (`int`, `long`, `long long`) presents the *dual* category-theoretic error to its unsigned sibling. Where `unsigned int` claims *more* structure than $\mathbb{N}$ has, `int` claims *less* structure than $\mathbb{Z}$ has.[8] The Kleene-truth-value version of the embedding chain is $\mathbb{K}\not\models \hookrightarrow$ `std::signed_integral` $\to \mathbb{Z}$, factored exactly as the Boolean / unsigned chain is: `embed_𝕂3_ℤ_` encodes the three Kleene values into the signed-int machine layer (`False` $\mapsto -1$, `Unknown` $\mapsto 0$, `True` $\mapsto 1$; post-PR #439), and the lift `std::signed_integral` $\to \mathbb{Z}$ (called `embed_sint_ℤ_`, post-issue #418 / renamed under #457) lands in the variant $\mathbb{Z}$-proxy.[9]

---

[6] The mod-wrap arithmetic gives every element an additive inverse: $u + (2^w - u) \equiv 0 \pmod{2^w}$. The textbook $\mathbb{N}$ is a commutative semiring *without* additive inverse; calling `unsigned int` "natural numbers" is a category-theoretic error in the direction of claiming *more* structure than $\mathbb{N}$ actually has.

[7] This framing is the correct reading of the project's *Honest Rejection* policy: a hardware species is rejected as a witness for an idealised mathematical structure when its laws differ from those of the idealised structure — and the rejection is symmetric in direction, so that "too much structure" (`unsigned int`'s modular inverse) is rejected on the same footing as "too little" (`int`'s undefined-behaviour overflow that defeats closure).

[8] The literal C++ ring operators ($+$, binary $-$, unary $-$, $*$) all close on `int` and the order layer is fine; the operator surface really is there. But *signed-overflow is undefined behaviour*, which means closure-under-arithmetic fails as a stable axiom. The library's posture is therefore honest in both directions: `HasRingOperators<int>` fires (the surface is real); `IsRing<int, std::plus, std::multiplies>` does not (the axiomatic-ring witness would be a false claim under UB). The exact-$\mathbb{Z}$ reading lives one carrier step deeper, on `SignedCardinality` (with $\pm\aleph_0$ saturation and `NaZ`).

[9] Both rejections are symmetric in form: each carrier's claim is mismatched with the textbook structure it would purport to inhabit, and the type system refuses the mismatch in both directions. An engineer who claims that the carrier their code uses is a ring "in spirit" is making the kind of public statement the truthfulness canons of [22] are about; the type system's refusal to accept the `IsRing<int>` witness is the mechanical analogue of an external review catching the claim.

5.2 Linear Programming: The Optimum as a Typed Constant

Stare at a linear program for a moment. The polytope is an intersection of halfspaces—a set, in exactly the comprehension sense of Section 4. The objective is a linear functional. The optimum is the one vertex of the polytope at which the objective is largest.

The previous section's reductions collapsed a set to a simpler set. An optimum is also a collapse: it reduces a set to a single point. The move of this section is to lift that collapse into the type system. If every halfspace and every objective coefficient is named at the type level, the compiler has enough information to enumerate the vertices, run the feasibility checks, compare the objective values, and emit the winner as a literal. The LP solver is the C++ compiler.

Showcase 9 (centrepiece). Consider a textbook LP:

$$\begin{aligned} \text{maximise} \quad & 3x + 2y \\ \text{subject to} \quad & x + y \leq 4 \\ & 2x + y \leq 6 \\ & x, y \geq 0 \end{aligned}$$

The feasible region is a polygon in $\mathbb{Q}^2$ (Figure 4); the optimum is the vertex $(2, 2)$ where the non-axis-aligned constraints $x+y = 4$ and $2x+y = 6$ are simultaneously binding (objective value 10). In `dedekind.optimization:lp`, constraints are NTTPs and the reduction is a variadic template whose output carries the vertex at the type level:

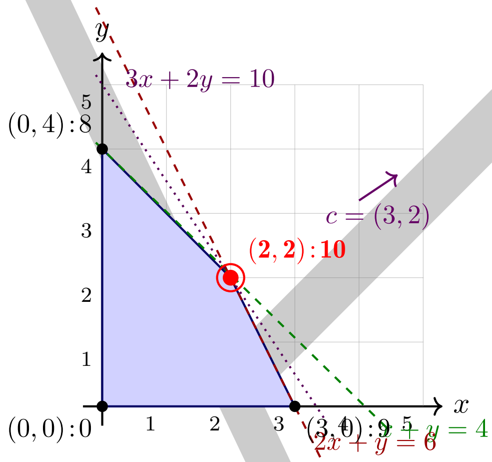

Figure 4: The feasible polygon of Showcase 9. Vertices are labelled with the objective value $3x+2y$ at each. The active-set enumeration considers all $\binom{4}{2} = 6$ pairs of constraints; four pairs produce feasible intersections, and the optimum is the one vertex (red) where both non-axis-aligned constraints $H_1\colon x+y \leq 4$ and $H_2\colon 2x+y \leq 6$ bind simultaneously. At translation time the compiler performs this enumeration; the emitted binary contains only the literal coordinates of $(2, 2)$.

```
using Rat = Rational<SignedExtensionalCardinal<>>;  // canonical exact ℚ
using H1 = Halfspace2D<Rat, Rat{1}, Rat{1}, Rat{4}>;     //  x +  y <= 4
using H2 = Halfspace2D<Rat, Rat{2}, Rat{1}, Rat{6}>;     // 2x +  y <= 6
using H3 = Halfspace2D<Rat, Rat{-1}, Rat{0}, Rat{0}>;    //  x       >= 0
using H4 = Halfspace2D<Rat, Rat{0}, Rat{-1}, Rat{0}>;    //       y >= 0

using Optimum = decltype(maximize<Rat, Rat{3}, Rat{2}, H1, H2, H3, H4>());
```

```
static_assert(std::same_as<Optimum, Vec2<Rat, Rat{2}, Rat{2}>>);
```

Listing 15: Compile-time LP: the optimum IS a type. From `showcase_09_lp_vertex_typed_constant.cpp`.

The reduction enumerates vertex candidates at compile time (all $\binom{4}{2} = 6$ pairs of binding constraints), solves each 2×2 active-set system via `Invertible2x2`'s closed-form Cramer inverse (exact over $\mathbb{Q}$), filters by feasibility against the non-active constraints, and picks the objective-maximising feasible vertex. Every step is a template specialisation; no runtime iteration survives.

IR witness. The paper's claim is not prose; it is translation-time mechanics. Two `extern "C"` functions in the fixture return the numerator of the optimum's x- and y-coordinates. At `-O2`, `clang`'s optimiser folds the entire LP reduction to literal integers:

```
define noundef i64 @witness_lp_optimum_x() ... {
  ret i64 2
}

define noundef i64 @witness_lp_optimum_y() ... {
  ret i64 2
}
```

Listing 16: Emitted LLVM IR for Showcase 9. The six candidate solves, six feasibility checks, and six objective comparisons collapse to constant returns.

The IR is checked into the repository as a fixture (`showcase_09_lp_vertex_typed_constant.ll`); a build-time gate (`make ir-fixture-check`) fails CI if optimiser drift ever loses the collapse. The binary contains no LP solver; no simplex tableau; no active-set machinery. The vertex *is* a type, and returning its coordinate *is* a literal. Because the reduction lands in `Rational<...>` rather than IEEE floating point, the `ret i64 2` output is bit-identical across ARM and x86 back-ends and host floating-point ABIs — the typed constant has no rounding mode to disagree about. This is the embedded / cross-platform-determinism guarantee the paper has been claiming throughout, made concrete by the absence of any `double`-typed reduction in the IR.

This closes one of the type-directed-collapse axes listed in Section 5.6 in a worked form: the LP solver is the C++ compiler, for every instance whose size and coefficients are compile-time data.

5.3 Forward-Mode AD as a Sibling Realisation of Compile-Time Derivatives

Swap the ring: re-run the same LP reduction over $\mathbb{Q}[\varepsilon]/(\varepsilon^2)$ instead of $\mathbb{Q}$. The optimum and its derivative with respect to a chosen input come out as one typed constant; no separate derivative pass. We follow Elliott's framing [7] that forward-mode AD is the chain-rule-carrying product (f, f'), with the dual-number ring making the chain rule fall out of the ring laws.[10]

Showcase: parametric LP sensitivity. Perturb H_1's bound by ε (replace $x + y \leq 4$ with $x + y \leq 4 + \varepsilon$). The active set is unchanged for infinitesimal perturbation; the Cramer solve now runs over duals:

```
using D = Dual<Rat>;
using H1P = Halfspace2D<D, D{Rat{1L}}, D{Rat{1L}},
                        D{Rat{4L}, Rat{1L}}>;  // bound = 4 + epsilon
```

[10]The same abstract derivative is computed numerically by `dedekind.analysis:ftc::derivative_at` (central difference) and symbolically by `dedekind.algebra:polynomial::RigPolynomial::derive`. The `Dual<F>` carrier slots into any NTTP-parameterised carrier —including the LP's `Halfspace2D<T, a, b, c>`—without special-casing, because its primal and tangent are structural fields.

```
using H2D = Halfspace2D<D, D{Rat{2L}}, D{Rat{1L}}, D{Rat{6L}}>;
using H3D = Halfspace2D<D, D{Rat{-1L}}, D{Rat{0L}}, D{Rat{0L}}>;
using H4D = Halfspace2D<D, D{Rat{0L}}, D{Rat{-1L}}, D{Rat{0L}}>;

constexpr auto v = maximize_value<D, D{Rat{3L}}, D{Rat{2L}},
                                  H1P, H2D, H3D, H4D>();
static_assert(v.x.val == Rat{2L}  && v.x.der == Rat{-1L});
static_assert(v.y.val == Rat{2L}  && v.y.der == Rat{2L});
```

Listing 17: Parametric LP over `Dual<Rat>`. From `lp_test.cpp`.

The primal $(2, 2)$ is the original optimum; the tangents $(-1, +2)$ match the hand derivation $x^*(t) = 2 - t$, $y^*(t) = 2 + 2t$ on the active set $\{x + y = 4 + t,\ 2x + y = 6\}$. The LP reduction was written once over an unconstrained ring-like carrier; instantiating it over the dual-number ring discharges sensitivity analysis as a side effect of the ring laws. The same `Dual`∘`Complex`∘`Rational` layering extends to holomorphic automatic differentiation at compile time; we leave its application to a follow-up.

5.4 Linear Algebra: $\mathbb{C}$ and $\mathbb{D}$ as Subrings of $M_2(\mathbb{Q})$

The classical regular representations $a+bi \mapsto \left(\begin{smallmatrix} a & -b \\ b & a \end{smallmatrix}\right)$ and $a+b\varepsilon \mapsto \left(\begin{smallmatrix} a & b \\ 0 & a \end{smallmatrix}\right)$ are ring monomorphisms $\mathbb{C}, \mathbb{D} \hookrightarrow M_2(\mathbb{Q})$ (see, e.g., Lang [30]). The library claims both identities at the type level over `Rational<SignedExtensionalCardinal<>>` and the compiler discharges them as bit-perfect `static_assert`s, with the defining relation $\varepsilon^2 = 0$ lifting to $\left(\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right)^2 = 0$ on the matrix side.

```
using Rat = Rational<SignedExtensionalCardinal<>>;
constexpr Complex<Rat> z{Rat{1}, Rat{2}}, w{Rat{3}, Rat{-5}};
constexpr Dual<Rat>    d{Rat{2}, Rat{3}}, e{Rat{-1}, Rat{5}};
static_assert(as_matrix2x2(z + w) == as_matrix2x2(z) + as_matrix2x2(w));
static_assert(as_matrix2x2(z * w) == as_matrix2x2(z) * as_matrix2x2(w));
static_assert(as_matrix2x2(d + e) == as_matrix2x2(d) + as_matrix2x2(e));
static_assert(as_matrix2x2(d * e) == as_matrix2x2(d) * as_matrix2x2(e));
constexpr Dual<Rat> eps{Rat{0}, Rat{1}};
static_assert(as_matrix2x2(eps) * as_matrix2x2(eps) == zero_matrix2x2_v<Rat>);
```

Listing 18: Ring-homomorphism identities $\mathbb{C}, \mathbb{D} \hookrightarrow M_2(\mathbb{Q})$, discharged as `static_assert`s; the source uses Catch2 `STATIC_CHECK` macros and `..LRat`. long-suffixed rationals, abridged here for legibility.

5.5 Algebraic Properties Enable Stronger Pruning

Beyond interval arithmetic, *algebraic* properties of the predicates provide additional pruning opportunities. The `:species` registry records which operations are associative, commutative, idempotent, and so forth. These traits inform the compiler about structural relationships between sets:

- $S \cap \varnothing = \varnothing$ (intersection with the empty set is empty).
- $S \cap \overline{S} = \varnothing$ (intersection with the complement is empty).
- If $S_1 \subseteq S_2$ then $S_1 \cap S_2 = S_1$ (subset absorption).

Each of these equalities can be discharged as a `static_assert` once the relevant algebraic witnesses are in the trait registry, reducing the intersection to the simpler set at compile time.

5.6 From Cardinality Pruning to Type-Directed Collapse

Step back from the showcases for a moment. What is the pattern? In every one of them we started with an intensional set—a predicate—and we ended with a named extensional object whose *type* carries the answer: $\varnothing$, a singleton, an order interval of known cardinality, an LP vertex. The collapse is not incidental; it is a rewrite rule, running inside the type system, at translation time. To make the claim precise, we first formalise "how much the compiler knows about a set" as a lattice, then state the theorem as monotone motion up that lattice.

Definition 1 (Computability tier)**.** *For a compile-time set S over an ambient carrier T, the* computability tier *is the triple*

$$\mathcal{T}(S) \;=\; \big(\mathsf{R}(S),\ \mathsf{L}(S),\ \mathsf{K}(S)\big) \;\in\; \{\mathrm{Int}, \mathrm{Ext}\} \times \{\mathrm{K3}, \mathrm{Cl}\} \times \{\mathrm{Opq}, \mathrm{Elt}\}$$

whose components are the representation *tier (R: intensional vs. extensional, i.e. predicate vs. elements-in-type), the* logic *tier (L: Kleene three-valued vs. classical two-valued membership), and the* knowledge *tier (K: elements opaque vs. carried in the type). Each axis carries the order* $\mathrm{Int} \preceq \mathrm{Ext}$, $\mathrm{K3} \preceq \mathrm{Cl}$, $\mathrm{Opq} \preceq \mathrm{Elt}$*; the product order is componentwise. The resulting Boolean lattice $(\mathcal{T}, \preceq)$ has bottom* $\bot = (\mathrm{Int}, \mathrm{K3}, \mathrm{Opq})$ *("a raw predicate over a floating-point carrier with no inhabitants known") and top* $\top = (\mathrm{Ext}, \mathrm{Cl}, \mathrm{Elt})$ *("a named finite set whose members are template parameters and whose membership is decidable").*

Every showcase of Section 5 is a motion inside this lattice. The empty-halfspace reductions of Showcase 3 go $(\mathrm{Int}, \mathrm{K3}, \mathrm{Opq}) \mapsto \top$ in one step (the result is $\varnothing$); the cardinality-1 reduction of Showcase 4 and the LP reduction of Showcase 9 do the same. The reductions never slide downward. With that in hand:

Theorem 1 (Type-Directed Collapse, totally-ordered 1D case)**.** *Let T be a totally ordered carrier, and let*

$$P_i(x) \;=\; (x \geq a_i), \quad i = 1, \ldots, m, \qquad Q_j(x) \;=\; (x \leq b_j), \quad j = 1, \ldots, n,$$

be lower-bound and upper-bound halfspace predicates over T whose pivots $a_i, b_j \in T$ are carried at the type level as non-type template parameters. Define the effective bounds $a^\star := \max_i a_i$ if $m \geq 1$ and $a^\star := \bot$ if $m = 0$, and $b^\star := \min_j b_j$ if $n \geq 1$ and $b^\star := \top$ if $n = 0$ (where $\bot, \top$ denote the carrier's bottom and top elements, taken as $\pm\infty$-style sentinels when T is unbounded). Then the meet

$$S \;=\; \bigcap_{i=1}^{m} \{x \in T \mid P_i(x)\} \,\cap\, \bigcap_{j=1}^{n} \{x \in T \mid Q_j(x)\}$$

is resolved at compile time to exactly one of:

1. *$\varnothing$, when $a^\star > b^\star$;*
2. *a `Singleton<v>`, when T is integral and $a^\star = b^\star = v$;*
3. *an `OrderInterval` $[a^\star, b^\star]$ otherwise.*

Proof. By induction on $m + n$.

Base case ($m + n = 0$): no constraints; $S = T$, which is the order interval $[\bot, \top]$ (case 3) by the boundary convention.

Inductive step. Assume the claim holds for k total constraints. Add one more; without loss of generality (by the symmetry between lower and upper bounds) take it to be a fresh lower-bound predicate $P_*(x) = (x \geq a_*)$. The inductive hypothesis gives a reduced form S_k of one of the three shapes. Form $S = S_k \cap \{x \mid x \geq a_*\}$. Two compile-time comparisons of a_* against the bounds of S_k (where present) suffice:

- **Redundant.** If S_k has lower bound a and $a_* \leq a$, then $\{x \mid x \geq a_*\} \supseteq S_k$ and $S = S_k$; the form is preserved.
- **Tightening.** If S_k is the interval $[a, b]$ (or unbounded above) and $a < a_* \leq b$ (or no upper bound), then $S = [a_*, b]$ in case 3; if T is integral and $a_* = b$, then $S = \{a_*\}$ (case 2).
- **Contradicting.** If S_k has upper bound b and $a_* > b$, or if S_k is already $\varnothing$, then $S = \varnothing$ (case 1).

The same case-split, with rôles of a_i and b_j exchanged, discharges the symmetric upper-bound addition. Each case lands in one of the three forms, completing the induction. $\square$

Theorems for free. The structural shape of Theorem 1 is Wadler's *Theorems for Free!* [3] move at the algebraic register. Wadler showed that polymorphic types in the typed lambda calculus discharge a class of theorems by parametricity alone, with no further proof obligation on the caller. Here, the analogous discharge runs from *carrier-axiom declaration* (`HasTotalOrderOperators<T>` + `IsTotallyOrdered<T>`, plus the integral-carrier witness when case 2 is to fire) to a carrier-level reduction of any finite intersection of half-line constraints. The engineer's honesty obligation is the axiom set; the collapse is type-level free. The proof above is one demonstration that the discharge is mechanical—the inductive argument does not depend on which carrier or which pivot type, only on the axiom slots being filled.

Existential anchor. The theorem is realised in the library by witnesses that materialise at the static-assertion layer; the canonical exhibit on $\mathbb{N}$ is the typed-constant collapse

$$\{x \in \mathbb{N} \mid x \geq 2\} \cap \{x \in \mathbb{N} \mid x \leq 5\} \;\longrightarrow\; \texttt{OrderInterval}\langle 2, 5\rangle,$$

checked at compile time. Section 5 catalogues these witnesses; the LLVM IR fixtures pin the resulting machine-code collapse downstream of the type-level reduction.

Conjecture 1 (Type-Directed Collapse, n-dimensional generalisation)**.** *Let $T_1, \ldots, T_d$ be totally ordered carriers and let $\{P_k\}_{k=1}^{N}$ be halfspace predicates over the product $T_1 \times \cdots \times T_d$ whose pivots are carried at the type level. The meet*

$$S \;=\; \bigcap_{k=1}^{N} \{(x_1, \ldots, x_d) \mid P_k(x_1, \ldots, x_d)\}$$

admits a compile-time reduction to a typed normal form generalising the three cases of Theorem 1 (empty, vertex-singleton, or a polytope record carrying its active constraint set), under a suitable axiom set on the product carrier (total order per axis, closure of the meet under the product, and a vertex-enumeration discipline for the polytope case). Writing S_k for the single-constraint set, the reduction is monotone up the computability-tier lattice:

$$\mathcal{T}(S) \;\succeq\; \bigvee_{k} \mathcal{T}(S_k),$$

and strictly so on at least one axis in every case the library exhibits.

Status of claim. *This conjecture is* not *proved in the paper. It is supported by the same exhibition pattern as the 1D theorem: each case is checked into the library as a `static_assert`/`STATIC_CHECK` witness and, where relevant, as an LLVM IR fixture that fails CI on optimiser drift (see Section 5). The 2D linear-programming showcase of Section 5.2 discharges one non-trivial instance via active-set vertex enumeration; a general inductive proof over pivot configurations and dimensions d is left open.*

Corollary 1 (Iterated collapse)**.** *Any finite composition of type-directed-collapse reductions is itself monotone up $\mathcal{T}$. In particular, no sequence of library operations can ever return a set that is* less *decidable than its inputs.*

So: not "a faster way to do set algebra", but a rewrite rule that happens to live in the C++ type system, and whose action is monotone-ascending in a lattice of decidability. The halfspace case is the smallest non-trivial instance we know how to prove mechanically. Four further axes appear to obey the same pattern; the project tracker carries one existential proof per axis—one worked instance, one IR fixture, one compile-time witness—rather than a general framework. We would rather ship one reduction that fully collapses than four sketches that do not.

- **Carrier tightening.** Intersecting a subset of $\mathbb{N}$ with a subset of $\mathbb{R}$ should land in $\mathbb{N}$: the carrier lattice contributes its own structural reduction via the existing canonical embeddings (`embed_uint_sint_`, `embed_ℚ_ℝ`, ...).
- **Smooth optimization.** The stationary set $\{x \mid \nabla f(x) = 0\}$ for a quadratic f collapses to a `Singleton<x*>` with the minimizer in the template parameter. Same reduction pattern, applied at the extremum operator.
- **Linear programming** (demonstrated in Section 5.2 and Section 5.3). `maximize(c, polytope)` reduces, for compile-time-sized LPs, to a typed vertex via type-level active-set enumeration, and the same reduction over a dual-number carrier yields sensitivity analysis with no extra code path: the solver is the compiler.
- **Lattice-law rewriting.** Distributivity and absorption applied at the type level rewrite $(A \cup B) \cap \neg A$ to $B \cap \neg A$ *before* local reduction rules fire, exposing collapses that a purely local analysis would miss.

Each axis sits on the foundation shared with Section 5: NTTP predicates, structured `operator&` dispatch, and the computability tiers visible in the showcases. Taken together they sketch a program of which the present halfspace reductions are one installment.

The polynomial-calculus exhibit (companion-repository `:polynomial` / `:ftc` partitions) belongs to the same family but at the *function* layer: a polynomial reduced under `derive` (a ring homomorphism) or a numerical integrator yields another intensional object whose identity with a target is discharged by the compiler.

5.7 The Compiler Wall: Where This Stops

Treating `clang++` as a poor man's theorem prover is useful precisely because the compiler is bounded, predictable, and cheap. It is also, for the same reason, incapable of doing everything a real proof assistant would. The purpose of this subsection is to be honest about the wall and to name the `clang` knobs that push it back by a constant factor but cannot remove it.

Template instantiation depth. Every reduction in this paper is written as a variadic template whose depth grows with the number of type arguments. `clang`'s default instantiation-depth limit is 1024; on deep recursions (e.g. an LP with many halfspaces, or the cartesian-product carriers of Showcase 8) we see it earlier than expected because concepts and NTTP lambdas each add their own frames. The escape hatch is `-ftemplate-depth=N`; the `dedekind` build raises it to 2048 on the affected translation units. This buys depth linearly in N at the cost of compile time and resident memory, and it does not help when the recursion is superlinear in the instance size.

`constexpr` step and call-depth limits. `-fconstexpr-steps` (default $\sim$ 1,048,576 on recent `clang`) caps the number of evaluation steps a single `constexpr` computation may take; `-fconstexpr-depth` caps call recursion independently. Both matter here because the LP reduction performs its Cramer solves and feasibility checks inside `consteval` functions. A 2×2 LP with four constraints is comfortably under the default; a 2×2 LP with thirty constraints ($\binom{30}{2} = 435$ active sets) begins to approach the limit with any nontrivial constraint metadata. Raising the cap is a compile-time cost; the more principled answer—and the one we prefer—is to stop at the boundary and hand off to a runtime solver, which is exactly the case our centrepiece is *not*.

Combinatorial blowup is the pessimistic bound. Compile-time vertex enumeration over n halfspaces is, in the worst case, $\binom{n}{d}$ active-set solves for a d-dimensional LP; in the 2D case this paper demonstrates, $\binom{n}{2}$. For $n = 4$ that is six solves; for $n = 20$, 190; for $n = 1000$, nearly half a million. Taken at face value the figure looks alarming. Two facts about real constraint lists soften it substantially.

First, by the fundamental theorem of linear programming, the optimum of a d-dimensional LP is attained at a vertex with at most d binding constraints. In practice the active set is often significantly smaller than d—many vertices of a handwritten polytope bind only one or two of the stated constraints, and most of the syntactic constraint list is either slack at the optimum or logically redundant. The $\binom{n}{d}$ figure is the enumeration cost, not a measure of what any vertex actually witnesses.

Second, many redundancies are *statically* removable when constraints live at the type level. Two halfspaces with the same normal reduce at compile time to the tighter bound: `x > 5 && x > 7` collapses to `x > 7` (stricter pivot wins), `x > 5 && x < 2` collapses to $\varnothing$ (contradiction), and `x > 3 && x < 5` over an integral carrier collapses to `Singleton<4>`. The library's `structured_and` dispatch already performs exactly these reductions for 1D halfspaces (Showcases 3 and 4 are the empty-meet and cardinality-1 witnesses); the same dispatch pattern extends naturally to 2D halfspaces whose normals are linearly dependent. A structurally pruned constraint list is typically much smaller than its syntactic form, and the combinatorial term tracks the pruned count, not the raw one.

For the embedded regime this paper targets—MPC steps with a handful of physical constraints, sensor-fusion guards with a few axis-aligned bounds, parametric configuration LPs—the raw constraint count rarely exceeds a few dozen, and compile-time pruning typically drops it further. The method remains tractable for the instance regime it was built for. The intended partition is unchanged: compile-time reduction for problems whose structure is fixed in the type, runtime simplex or interior-point solvers (and the attendant numerical compromises) for problems whose size or coefficients are only known at execution. Industrial-scale LP is emphatically the latter; we do not claim it is the former. A genuine pathological case would require a constraint list that the author was *unable* to prune structurally—an unlikely shape for the use cases this library is written against.

Active-set hints as the user-side escape valve. Even within the compile-time regime, a user with domain knowledge of *which* halfspaces bind at the optimum can short-circuit the $\binom{n}{d}$ enumeration entirely: because the constraint pack is already an NTTP list, instantiating `maximize<T, cx, cy, H_i, H_j>()` with just the binding pair (instead of the full pack) is exactly the active-set hint, and the library evaluates only that single Cramer solve. An explicit `with_active_set<...>` API that takes the hint as a separate template argument and verifies feasibility against the full pack without re-enumerating is a natural successor surface, but the underlying NTTP-pack-as-hint mechanism is available today: dropping slack constraints from the syntactic list is the user's prerogative, and the combinatorial term then tracks the hinted count rather than the raw one.

Scaling beyond atomic carriers via structure-preserving combinators. The same partition—atoms below, runtime above—governs the linear-algebra surface the paper exhibits at the 2×2 scale. The library ships `Invertible2x2`, `Matrix2x2V`, `Vec2V` as statically-validated atoms; larger systems are *not* reached by inflating the atom (we do not aim for $n \times n$ inversion in templates), but by composing atoms through *structure-preserving combinators* that propagate the invariants up. `DirectSum<A, B>` carries invertibility from its factors to the block-diagonal whole and is shipped today; 4×4 invertibility is therefore already a one-combinator step from the 2×2 atom, with $(A \oplus B)^{-1} = A^{-1} \oplus B^{-1}$ exact over $\mathbb{Q}$ and verifiable by `static_assert`:

```
TEST_CASE("linear_algebra:matrix␣---␣Tier␣1:␣"
          "DirectSum␣preserves␣invertibility") {
  constexpr Invertible2x2<Rat, Rat{1L}, Rat{2L},
                              Rat{3L}, Rat{4L}> block_M{};
  // 90° rotation over ℚ.
  constexpr Invertible2x2<Rat, Rat{0L}, Rat{-1L},
                              Rat{1L}, Rat{0L}> block_rot{};
  constexpr DirectSum<decltype(block_M),
                      decltype(block_rot)> ds{};
  constexpr auto ds_inv = ds.inverse();

  using IdentityDirectSum =
      DirectSum<Identity2x2<Rat>, Identity2x2<Rat>>;
  STATIC_CHECK(ds * ds_inv == IdentityDirectSum{});
  STATIC_CHECK(ds_inv * ds == IdentityDirectSum{});
}
```

`BlockUpperTriangular<A, B, D>` extends the same propagation to triangular blocks under a degenerate-Schur condition (also shipped, also testable as a single `static_assert`); a future Kronecker combinator preserves orthogonality and lattice structure compositionally. Each combinator is itself a small structurally-checked unit; their composition gives $2^k \times 2^k$ matrices whose invariants follow by induction without exponential blowup in the templates. The remaining roadmap items fall out of the same Axiomatic-Systems-Programming discipline rather than requiring a different paradigm, and are tracked in the project backlog rather than discharged here. The $2 \times 2 \to 4 \times 4$ step is a witness, not a promise; everything beyond is the natural recursion of the same combinator pattern.

Numeric overflow in the carrier. The carrier-vs-overflow trade-offs are catalogued in Section 3.4's Rosetta tables: showcases default to `Rational<SignedExtensionalCardinal<>>` (exact, no silent overflow); programmers who trade exactness for native performance pick the corresponding row of Table 2 and accept the named failure mode (signed-overflow UB on `int`/`long`; non-associative rounding on `float`/`double`). Every combinator in Section 5 is written against `HasRingOperators` rather than any particular carrier, so the swap is a one-line change at the `using` declaration and the compile-time-reduction-to-a-typed-constant claim survives every row of the Rosetta.

Error messages and the readability tax. A failed `static_assert` in a deep reduction produces, by default, a multi-page expansion trail that is difficult for a new user to parse. The bulk of this is a C++ language-design concern that no library can fully repair; the realistic ambition is to *fail fast*, so that the first message a reader sees is the predicate that genuinely broke and not the cascade of downstream failures it triggered. The mitigation the library adopts is exactly that: `requires`-clauses at call sites (so the first failing predicate is the reported one), named concepts with short `diagnostic` strings, targeted `static_assert` messages inside internal reductions, and—structurally—building combinators out of statically-validated atoms so that a failure is localised to its smallest witness rather than surfacing inside a deep template instantiation. The `IsRing` / `IsField` / `IsCyclicGroup` concepts of `dedekind.category:total` all follow this pattern: each adds a single named requirement on top of a previously-established one, so the first violated predicate is also the most specific one. This is an ergonomics concern rather than a correctness one, but it is the concern a reader most often encounters first and deserves explicit acknowledgement.

Worked diagnostic. A rank-deficient `Invertible2x2<Rat, 1, 2, 2, 4>` input produces a single named diagnostic naming the broken predicate (`Invertible2x2 requires full rank: a*d - b*c != 0`) at the `static_assert` site, with one `note` pointing at the instantiation; no substitution-failure cascade. The predicate sits inside the class body as a literal `static_assert` rather than as a `requires`-clause that has to be probed, so the diagnostic fires at the named axiom rather than as a probe failure deep in overload resolution. This is *fail fast at the named axiom*: the mathematical claim that broke surfaces before the type-system mechanics that surfaced it.

In short: the compiler wall has a predictable shape—depth, steps, combinatorics, numeric range, diagnostics—and each is either pushable by a flag, avoidable by staying within the instance regime for which this technique is intended, or a known limitation we do not claim to have solved. The paper's contribution lies entirely to the left of that wall.

5.8 Boundary Observations: An Axiom → Theorem Chain

The previous subsections present the technique through worked showcases, IR artefacts, and concrete diagnostics—an empirical register. The library admits a stricter framing whose structural parallel is Wadler's *Theorems for Free!* [3]: parametricity gives the typed-lambda calculus theorems from type signatures alone; the dedekind library extends the move into *algebraic* territory, where a carrier's axiomatic profile (declared via opt-in traits and concept witnesses) discharges carrier-level theorems at the type level. The empirical artefacts of the showcases are then *realisations* of an underlying axiom → theorem chain rather than the substance of the chain itself.

Axioms as the engineer's honesty obligation. A carrier's algebraic profile is asserted explicitly. Where the proof of an axiom is beyond what C++ concepts can quantify, the claim is made via an opt-in trait specialisation, and the engineer who declares the trait carries Wadler's *propositions-as-types* [4] obligation under the NSPE Code of Ethics canons III and IV (truthful public statements; faithful agency). The library ships several such opt-in surfaces: `is_monic_arrow_v` for monomorphism declarations, `is_initial_ring_v` for the initial-object reading of $\mathbb{Z}$, `is_grothendieck_group_v` for the free-abelian-group reading, `is_initial_f_algebra_v` for the NNO-as-initial-F-algebra reading, and a family of axiom-shape concepts including `IsRing`, `IsField`, `IsTotallyOrdered`, `IsArchimedean`, and `IsCyclicGroup`. Each is the engineer's claim, not a machine-discharged proof; the structural shape (signatures, closure, opt-in trait set to `true`) is what the type system can pin mechanically.

Theorems for free. Once the axioms are discharged honestly, downstream code leans on a catalogue of carrier-level theorems without further proof: generic combinators over `HasRingOperators<T>` compose for any carrier whose ring axioms hold; the carrier-swap `Rational`⟨`long`⟩ → `Rational`⟨`SignedExtensi` is a one-line change at the `using` declaration and every section's claim survives the swap; the type-directed collapse of Section 5 (whose 1D specialisation admits a worked proof, deferred to a follow-up paper) reduces a constraint intersection to one of $\varnothing$, `Singleton<v>`, or a typed `Interval` entirely from carrier-level total-order + meet-closure; and the partition naming (`:natural` / `:integer` / `:rational` / `:real` / `:complex`) lifts the textbook vocabulary into the type system unchanged. The trade-off is plain: *if* a carrier's domain admits the axioms, the affordances pay;

if the domain hits a friction (next paragraph), the axioms cannot be discharged honestly and downstream code falls back to the operational policy gate.

Frictions: where C++23 and IEEE numerics break the chain. The axioms cannot always be discharged. The friction points encountered while building the library are observational, not advocacy, and each is pinned at a concrete file in source (with the relevant lines flagged in the corresponding partition's witness blocks) so a reader can verify the mechanism rather than take the claim on faith. *Integer-promotion shape mismatch*: `bool + bool` and `bool * bool` return `int` rather than `bool`; `unsigned short + unsigned short` / `unsigned short * unsigned short` and `unsigned char + unsigned char` / `unsigned char * unsigned char` likewise promote to `int`. The functor surface (`std::plus<bool>`, `std::multiplies<unsigned short>`) returns the carrier by signature, but the literal `+` / `*` operator surface does not (witnessed at `algebra/ring.cppm`, `algebra/field.cppm`, `numbers/uint.cppm`, `order/lattice.cppm`). *Signed-overflow UB defeats* $\mathbb{Z}$: `int` / `long` / `long long` carry the literal ring operator surface (`HasRingOperators<int>` fires) but *not* the axiomatic ring witness, because closure-under-arithmetic fails as a stable axiom under signed-overflow UB; the negative pins `! Group_<int>`, `!IsArithmeticAdditiveGroup<int>`, `!category::IsRing<int, std::plus, std::multiplies>` at `numbers/sint.cppm` make the rejection mechanical. *IEEE 754 breaks reflexivity*: `NaN <= NaN` evaluates to `false`, so `is_reflexive_v<double, std::l` is deliberately not registered, and `!IsTotallyOrdered<double>`, `!IsTotallyOrdered<long double>`, and `!category::IsRing<double, std::plus, std::multiplies>` all hold (`numbers/floating_point.cppm` + `order/total.cppm`); the operational ordered-field reading survives via `IsOrderedField<double>` in `morphologies/archimedean.cppm`. *`std::variant` NTTP-unfriendliness*: the variant $\mathbb{Z}$-proxy `SignedCardinality` (variant of `SignedExtensionalCardinal<>`, $\pm\aleph_0$, `NaZ`) cannot be a non-type template parameter without workarounds, blocking compile-time arithmetic on the saturating integer carrier (`sets/cardinality.cppm`, `numbers/rational.cppm`). *`std::integral` unifies three structurally distinct algebras*: `std::integral T = std::unsigned_integral T || std::signed_integral T` and the three siblings are a finite cyclic ring, an operator-surface-without-axioms carrier, and (separately) the Boolean rig + Galois field $\mathbb{F}_2$; nothing axiomatic survives the union (`numbers/integral.cppm`). Each friction is the structural reason an axiom slot is left *unsatisfied* for the relevant family.

The policy gate as the bridge. Where the strict axioms cannot be discharged but the operator surface must still be programmed against, the library exposes an operational policy gate. The canonical example is the rational-field claim, which the showcases program against without committing the carrier to a specific machine integer:

```
// numbers/rational.cppm --- operational field-like witness for ℚ.
// IsFieldLikeScalar is the operator-surface gate; the strict
// axiomatic field witness lives at category::IsField but is not
// load-bearing for downstream code that only needs +, -, *, /.
static_assert(
    dedekind::algebra::IsFieldLikeScalar<Rational<default_integer>>,
    "Rational<I>␣must␣satisfy␣the␣operational␣field-like␣witness␣"
    "(ℚ␣is␣a␣field).");

static_assert(
    dedekind::algebra::IsFieldLikeScalar<
        Rational<SignedExtensionalCardinal<>>>,
    "Rational<SignedExtensionalCardinal<>>␣must␣satisfy␣"
    "IsFieldLikeScalar:␣the␣intended␣arbitrary-precision␣"
    "signed-rational␣carrier␣for␣ℚ.");
```

The two carriers differ in their machine-level friction profile (`default_integer` trips signed-overflow UB on extreme inputs; `SignedExtensionalCardinal<>` does not, at the cost of variant-NTTP friction) but both discharge the operational gate. Downstream combinators bind to

`IsFieldLikeScalar` rather than to `category::IsField`, and the carrier swap between the two is a one-line change at the `using` declaration. The gate is the load-bearing concept; the strict axiomatic field is reachable, on demand, when a use site warrants the heavier obligation.

In short, where Section 5.7 names the *mechanical* boundaries of the technique (template depth, `consteval` budgets, combinatorial blowup, diagnostics), this subsection names the *algebraic* boundary: which axioms discharge cleanly on which carriers, where the obligation falls to the engineer, and which frictions block the discharge. The empirical artefacts of the showcases sit one level below this chain—they demonstrate that the library realises the axiom $\to$ theorem discipline in C++23 today, on the carriers in the Rosetta tables of Section 3.4.

6 Related Work

Functional and dependently-typed approaches. Agda [9] and Coq provide dependent type systems in which set-membership proofs are first-class terms. The expressiveness is greater than C++23, but the runtime model differs fundamentally: proof terms are erased at runtime, whereas `dedekind`'s structural pruning targets machine-code generation in a systems-programming context. Haskell [8] supports type-class-based structural reasoning, and libraries such as `algebra` encode ring and field hierarchies. C++ templates differ in that they operate on concrete machine types (with hardware-level algebraic imperfections) rather than erased polymorphic dictionaries.

Exact arithmetic. The GNU Multiple Precision library (GMP) and `mpfr` support arbitrary-precision arithmetic at runtime. `iRRAM` [31] and similar libraries implement computable real arithmetic. `dedekind` is complementary: exact reals (Dedekind cuts) are modelled *intensionally as types*, and realization to a specific arithmetic backend is an explicit programmer choice.

Set-builder type systems. Refinement types [32] attach predicates to types, enabling a form of set-builder representation in ML-family languages. Liquid Haskell [33] extends Haskell with SMT-backed refinement predicates. `dedekind` achieves a subset of this functionality in standard C++23 without a custom type checker, using NTTP-embedded lambdas as the predicate carrier.

Linear programming. The standard LP toolchain—simplex [34] and interior-point methods [35] as implemented in Gurobi, CPLEX, GLPK, and related solvers—treats the problem instance as runtime data: the objective vector, constraint matrix, and right-hand side are passed to a solver that iterates to a vertex. `dedekind` sits orthogonally: problem instances whose size and coefficients are *compile-time data* reduce via type-level vertex enumeration to the optimum as a typed constant (Section 5.2), with the entire solver collapsing out of the emitted binary. This is not a replacement for runtime LP—for large, data-dependent instances the classical pipeline remains indispensable—but a complement for instances that appear during code generation (problem-shape metadata, compile-time configuration polytopes, template-parametric optimizer settings). Exact-rational arithmetic over $\mathbb{Q}$ connects the approach to the exact-LP literature [36]; the specific contribution is the realisation at translation time with zero runtime overhead, witnessed in the emitted LLVM IR fixtures.

Automatic differentiation. Runtime AD libraries such as Ceres [12], Adept [37], Sacado, dco/c++, and CasADi [38] provide forward- and reverse-mode AD via expression templates or operator overloading on a specialised arithmetic type. The theoretical view taken by Elliott [7]—that forward-mode AD is the $(\text{value}, \text{tangent})$ product with chain-rule composition, not the dual-number object *per se*—clarifies why the same reduction that solves the LP also computes its sensitivities when instantiated over the right ring (Section 5.3). `dedekind`'s contribution is not a new AD algorithm but the observation that a single compile-time reduction, written once over an `HasRingOperators` carrier, serves as LP solver over $\mathbb{Q}$ and as forward-mode AD engine over $\mathbb{Q}[\varepsilon]/(\varepsilon^2)$ with no extra code path.

Embedded machine-learning runtimes and edge computing. The embedded / edge-computing [39] landscape has a mature runtime ecosystem that `dedekind` deliberately sits *beside* rather than competes with. TensorFlow Lite Micro [14] compiles models to a small-footprint interpreter targeted at microcontroller-class devices; ONNX Runtime and microTVM generate target-specific code from a model graph; the Eigen linear-algebra library is the de facto substrate for embedded numerics. These runtimes take the model as input and the target as fixed; they do not treat the hosting application's algebraic invariants as proof obligations for the C++ compiler to discharge.

A deliberate choice on the linear-algebra backend is worth flagging here, since it connects the library's position to the abstract-naturals-versus-IEEE-numerics tension of Section 3.2. Eigen [40] is mature, widely deployed, and excellent at what it targets—but it is also *carrier-opinionated*: its matrix types default to `float` or `double` and its optimisations assume a field-like IEEE carrier. A natural path for `dedekind`'s runtime linear-algebra layer is instead toward GraphBLAS [41, 42], whose explicit *(carrier, +, ×)* semiring parameterisation makes the ring-over-which-matrices-live a first-class configurable property. That matches the discipline this paper argues for: matrices over `bool` (with OR/AND) are perfectly well-defined objects when inversion is not required, matrices over `char` are useful for quantised embedded inference, and both sit naturally inside the `HasRingOperators` / `HasSemiringOperators` concept hierarchy. The cost of this generality is library complexity rather than carrier simplicity. The trade we advocate is correctness-and-performance (tight control over the ring structure) at the price of implementation effort, against Eigen's opposite trade—an ergonomic default carrier whose optimisations are largely locked to floating-point semantics.

`dedekind` addresses a different surface: the *inline* numerical primitives (Kalman filters, pose-estimation math, MPC steps, parametric optimisation) a robotics, automotive, or instrumentation engineer writes directly in their control loop. For these, a fixed-topology LP, a compile-specialised Kalman update, or an exact-rational sensor-fusion step expressed against `HasRingOperators` concepts exhibits three properties the model-runtime path does not aim at: a binary with no interpreter, a dependency graph no heavier than the system's `clang++`, and bit-for-bit deterministic arithmetic across platforms when the carrier is $\mathbb{Q}$. We see this as a complement to the runtime ecosystem, not a replacement: model-driven inference continues to belong to TFLite Micro and its siblings, while the numerical glue around such inference—the Kalman front-end, the control-loop LP, the gradient for online adaptation—is where Axiomatic Systems Programming is strongest.

Data-pipeline DSLs. Apache Spark, Flink, and the Python `pandas`/`polars` ecosystem provide declarative data pipelines with runtime domain checks. None of these ground their APIs in formal set theory, so intersections and filters are runtime pipeline operations rather than compile-time type invariants. `dedekind` is positioned one layer down: it is a systems-programming library that could in principle back such pipelines, but the contribution we demonstrate here is the compile-time reduction at the C++ layer itself.

C++ template metaprogramming lineage. Compile-time computation in C++ has a long tradition that `dedekind` builds on rather than supplants. Boost.Hana [10] and `mp11` [43] provide compile-time heterogeneous containers and type-level algorithm support; Hana in particular already demonstrates concept-based structural dispatch at compile time and would be a natural lower-layer substrate for several of our combinators. CTRE [11] shows that serious computation (regular-expression matching, compiled to a type-level automaton) can live entirely inside the type system and emit optimised machine code; the shape of that reduction—intensional description to a named extensional object at translation time—is close kin to our halfspace and LP reductions, differing in subject matter but not in mechanism. Expression-template AD libraries including ENOKI [13] and the runtime AD systems Ceres [12], Adept [37], and CasADi [38] have carried pieces of the dual-number story into production workloads.

`dedekind`'s position relative to this lineage is specific and narrow: it targets *algebraic-semantic* content—ring homomorphisms, structural cardinality, ordering laws, the (f, f') product as an Elliott [7] sibling—rather than the syntactic structure manipulation that is the traditional focus of TMP libraries. The contribution is not another metaprogramming trick but the claim, supported by worked reductions, that the *discipline* of treating mainstream C++23 as a structural-gate theorem prover (Axiomatic Systems Programming) is now coherent enough to practise end-to-end.

7 Conclusion

The thesis of this paper, once more: *named algebraic structure at the type level, discharged by the compiler, narrows the gap between abstract mathematics and machine code without a separate proof assistant.* `dedekind` is the existence proof. Sets are rules rather than buckets; intensional descriptions defer materialisation to explicit boundaries; the LLVM backend carries discharged proofs through to object-code optimisation. The centrepiece existential proof (Section 5.2) shows a textbook linear-programming instance reducing at compile time to its optimal vertex as a typed constant—the six candidate solves, six feasibility checks, and six objective comparisons vanish in the emitted IR. The same reduction over the dual-number ring recovers exact forward-mode sensitivities (Section 5.3); at the linear-algebra layer it certifies the ring homomorphisms $\mathbb{C}, \mathbb{D} \hookrightarrow M_2(\mathbb{Q})$ as compile-time facts (Section 5.4). The companion repository's polynomial-calculus partitions discharge the Fundamental Theorem of Calculus at translation time on the same discipline. In every case the binary contains only literals.

Axiomatic Systems Programming as a practised discipline. The viewpoint we name in Section 1—treating `clang++` as a poor man's theorem prover, and the `HasRingOperators` concept as a proof obligation the carrier must discharge—is the paper's intended export. Taken seriously, it turns each algebraic invariant into a structural gate the compiler polices; taken habitually, it shifts the library author's default from "runtime check" to "type-level obligation". Three axes of type-directed collapse remain open (Section 5.6): carrier tightening, smooth optimization, and lattice-law rewriting; each sits on the same NTTP-plus-concepts foundation as the halfspace and LP reductions demonstrated here. We leave them as worked targets for successor libraries that adopt the same discipline.

A forward claim, in its weakest defensible form. As numerical workloads saturate available hardware, the marginal returns of brute-force optimisation are diminishing, and the lever with a different cost curve is specialisation at translation time rather than run time. That lever also has a sustainability dimension: cycles not spent at run time are watt-hours not drawn from the grid, and the machine-learning research community has begun to measure its compute footprint in exactly those terms [44]. The operating point at which the lever compounds most visibly is not the data centre but the edge: CPU-first and embedded machine-learning deployments [39, 14], where the problem topology is fixed, the runtime footprint is under hard constraint, and the toolchain of least resistance is the system's `clang++`. We do not prove here that axiomatic discipline applied at library scale yields a meaningful fraction of cycles back at either end of the spectrum; we claim only that the ingredients—compile-time verification, structural optimisation, ring-aware code generation, exact arithmetic carriers—are now assembled in mainstream C++23, and that the discipline is worth practising precisely because its payoffs accrue at scales larger than any single library. If even a small fraction of the arithmetic done in contemporary edge deployments—model-predictive control in vehicles, Kalman-style sensor fusion in robotics, gradient evaluations in micro-inference loops—ran against libraries whose algebraic invariants had been discharged upstream, the compounded effect would be difficult to dismiss. The shape of that opportunity is what Axiomatic Systems Programming claims. Whether the shape materialises into the numbers it suggests is an engineering question, and one the next decade of library authors will answer.

Reproducibility under heterogeneous tooling. The discipline this paper names is, deliberately, falsifiable. The constraints under which it stays honest—textbook-anchored vocabulary, a named-module dependency graph, structural assertions discharged at translation time, a CI-policed test suite—do not depend on which agent typed any particular line of code, nor on the specific host language. As Section 2 notes, the technique transfers with modest adjustment to any strongly-typed-and-tested substrate whose type system can name a predicate over compile-time data and check it mechanically: Scala under refined types and a strict build, C# with constrained generics under Roslyn analysers, Java with sealed types, strictly- typed Python under `mypy` or `pyright`. C++23 is the worked example here—chosen because of the LLVM lever from discharged proof to optimised object code—but the discipline does not turn on it. A practitioner adopting the same constraints and a comparable toolchain in 2026 should be able to produce comparable artefacts on comparable timescales, irrespective of host language or of how the keystrokes were generated. The discipline is meant to outlast any particular generation of authoring tool—and, increasingly, of programming language.

References

[1] *<Programming>: The Art, Science, and Engineering of Programming.* `https://programming-journal.org/`. Open-access journal for programming-related research; ISSN 2473-7321.

[2] F. William Lawvere. “An elementary theory of the category of sets”. In: *Proceedings of the National Academy of Sciences* 52.6 (1964), pp. 1506–1511.

[3] Philip Wadler. “Theorems for free!” In: *Conference on Functional Programming Languages and Computer Architecture*. ACM. 1989, pp. 347–359.

[4] Philip Wadler. “Propositions as types”. In: *Communications of the ACM* 58.12 (2015), pp. 75–84. DOI: `10.1145/2699407`.

[5] The dedekind Authors. *dedekind: Axiomatic Systems Programming in C++23*. `https://github.com/vincentk/dedekind`. Open-source implementation, Apache-2.0 licensed. 2026.

[6] IEEE. *IEEE Standard for Floating-Point Arithmetic*. Provides the formal specification for floating-point formats and operations. IEEE. 2019. DOI: `10.1109/IEEESTD.2019.8766229`.

[7] Conal Elliott. “The simple essence of automatic differentiation”. In: *Proceedings of the ACM on Programming Languages* 2.ICFP (2018), pp. 1–29.

[8] The GHC Development Team. *The Glasgow Haskell Compiler User’s Guide, Edition 2024*. GHC 2024 Language Edition. 2024. URL: `https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/control.html`.

[9] Ulf Norell. “Towards a practical programming language based on dependent type theory”. PhD thesis. Chalmers University of Technology, 2007. URL: `http://www.cse.chalmers.se`.

[10] Louis-Dionne Chagnon. "Boost.Hana: Your Metaprogramming Nightmare". In: *CppCon 2015*. `https://github.com/boostorg/hana`. 2015.

[11] Hana Dusíková. "Compile-Time Regular Expressions". In: *CppCon*. `https://github.com/hanickadot/compile-time-regular-expressions`. 2018.

[12] Sameer Agarwal, Keir Mierle, and The Ceres Solver Team. *Ceres Solver*. `https://github.com/ceres-solver/ceres-solver`. 2022.

[13] Wenzel Jakob. "Enoki: Structured Vectorization and Differentiation on Modern Processor Architectures". In: `https://github.com/mitsuba-renderer/enoki`. 2019.

[14] Robert David et al. "TensorFlow Lite Micro: Embedded Machine Learning for TinyML Systems". In: *Proceedings of Machine Learning and Systems (MLSys)*. 2021.

[15] nLab authors. *structural set theory*. ncatlab.org/nlab/show/structural+set+theory. 2024. URL: `https://ncatlab.org`.

[16] Joachim Lambek and Philip J Scott. *Introduction to Higher-Order Categorical Logic*. Cambridge University Press, 1988.

[17] Jeff Bezanson et al. "Julia: A Fresh Approach to Numerical Computing". In: *SIAM Review* 59.1 (2017), pp. 65–98. DOI: `10.1137/141000671`.

[18] Wenzel Jakob. *nanobind: tiny and efficient C++/Python bindings*. `https://github.com/wjakob/nanobind`. 2022.

[19] Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, 2002. ISBN: 9780262162098.

[20] Luca Cardelli. "Type systems". In: *ACM Computing Surveys* 28.1 (1996), pp. 263–264. DOI: `10.1145/234313.234418`.

[21] William Shakespeare. *The Most Excellent and Lamentable Tragedy of Romeo and Juliet*. Act 2, Scene 2. John Danter, 1597.

[22] National Society of Professional Engineers. *NSPE Code of Ethics for Engineers*. Latest revision July 2019. NSPE Publication #1102. 2019. URL: `https://www.nspe.org/sites/default/files/resources/pdfs/Ethics/CodeofEthics/NSPECodeofEthicsforEngineers.pdf`.

[23] Roger Penrose. *The Road to Reality: A Complete Guide to the Laws of the Universe*. Chapter 16 and surrounding material on Gödel's incompleteness theorem. London: Jonathan Cape, 2004.

[24] Alan M. Turing. "Systems of Logic Based on Ordinals". In: *Proceedings of the London Mathematical Society*. Second Series 45 (1939), pp. 161–228.

[25] Zoran Majkić. "Intensional First-Order Logic for P2P Database Systems". In: *Journal on Data Semantics* 12 (2009), pp. 47–66. DOI: `10.1007/978-3-642-00684-6_4`. URL: `https://dblp.org/rec/journals/jods/Majkic09`.

[26] Harold Abelson, Gerald Jay Sussman, and Julie Sussman. *Structure and Interpretation of Computer Programs*. 2nd. The canonical exposition of *programming by wishful thinking*: name the procedure you wish you had, use it, then later supply its definition. MIT Press, 1996.

[27] F. William Lawvere. "An elementary theory of the category of sets". In: *Proceedings of the National Academy of Sciences* 52.6 (1964), pp. 1506–1511.

[28] Saunders Mac Lane. *Categories for the Working Mathematician*. Graduate Texts in Mathematics 5. New York: Springer-Verlag, 1971.

[29] F. William Lawvere and Robert Rosebrugh. *Sets for Mathematics*. Discusses the construction of the continuum and Dedekind cuts within a categorical framework. Cambridge University Press, 2003.

[30] Serge Lang. *Algebra*. 3rd. Vol. 211. Graduate Texts in Mathematics. Revised edition. Springer, 2002. ISBN: 9780387953854. DOI: 10.1007/978-1-4613-0041-0.

[31] Norbert Th. Müller. "The iRRAM: Exact Arithmetic in C++". In: *Computability and Complexity in Analysis (CCA 2000)*. Vol. 2064. Lecture Notes in Computer Science. Springer, 2001, pp. 222–252.

[32] Tim Freeman and Frank Pfenning. "Refinement Types for ML". In: *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, 1991, pp. 268–277.

[33] Niki Vazou et al. "Refinement Types for Haskell". In: *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. ACM, 2014, pp. 269–282.

[34] George B. Dantzig. *Linear Programming and Extensions*. Princeton, NJ: Princeton University Press, 1963.

[35] Narendra Karmarkar. "A New Polynomial-Time Algorithm for Linear Programming". In: *Combinatorica* 4.4 (1984), pp. 373–395.

[36] Ambros Gleixner, Daniel E. Steffy, and Kati Wolter. "Iterative Refinement for Linear Programming". In: *INFORMS Journal on Computing* 28.3 (2016), pp. 449–464.

[37] Robin J. Hogan. "Fast Reverse-Mode Automatic Differentiation Using Expression Templates in C++". In: *ACM Transactions on Mathematical Software* 40.4 (2014), 26:1–26:16.

[38] Joel A. E. Andersson et al. "CasADi: A Software Framework for Nonlinear Optimization and Optimal Control". In: *Mathematical Programming Computation* 11.1 (2019), pp. 1–36.

[39] Weisong Shi et al. "Edge Computing: Vision and Challenges". In: *IEEE Internet of Things Journal* 3.5 (2016), pp. 637–646.

[40] Gaël Guennebaud, Benoît Jacob, et al. *Eigen v3: A C++ Template Library for Linear Algebra*. https://eigen.tuxfamily.org. 2010.

[41] Jeremy Kepner and John Gilbert. *Graph Algorithms in the Language of Linear Algebra*. Software, Environments, and Tools. SIAM, 2011.

[42] Timothy A. Davis. "Algorithm 1000: SuiteSparse:GraphBLAS: Graph Algorithms in the Language of Sparse Linear Algebra". In: *ACM Transactions on Mathematical Software* 45.4 (2019), 44:1–44:25.

[43] Peter Dimov. *mp11: A C++11 Metaprogramming Library*. https://github.com/boostorg/mp11. 2017.

[44] Roy Schwartz et al. "Green AI". In: *Communications of the ACM* 63.12 (2020), pp. 54–63.